COLLEGES BRETAGNE LOIRE | SCIENCES POUR L'INGENIEUR ET LE NUMERIQUE

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

THÈSE DE DOCTORAT DE

L'ÉCOLE NATIONALE SUPÉRIEURE
MINES-TÉLÉCOM ATLANTIQUE BRETAGNE
PAYS DE LA LOIRE – IMT ATLANTIQUE

ÉCOLE DOCTORALE 648
Sciences pour l'Ingénieur et le Numérique
Spécialité : *Sciences et technologies de l'information et de la communication*

Par
Matthew COYLE

Exploration d'ensembles de modèles II

Object Oriented Constraint Programming

Thèse présentée et soutenue à IMT Atlantique Nantes, le « date »
Unité de recherche : « voir README et le site de de votre école doctorale »

Rapporteur·trice·s avant soutenance :

Prénom NOM	Fonction et établissement d'exercice
Prénom NOM	Fonction et établissement d'exercice
Prénom NOM	Fonction et établissement d'exercice

Composition du Jury :

Attention, en cas d'absence d'un·e des membres du Jury le jour de la soutenance, la composition du jury doit être revue pour s'assurer qu'elle est conforme et devra être répercutée sur la couverture de thèse

Président·e :	Prénom NOM	Fonction et établissement d'exercice *(à préciser après la soutenance)*
Examinateur·trice·s :	Prénom NOM	Fonction et établissement d'exercice
	Prénom NOM	Fonction et établissement d'exercice
	Prénom NOM	Fonction et établissement d'exercice
	Prénom NOM	Fonction et établissement d'exercice
Dir. de thèse :	Samir LOUDNI	Fonction et établissement d'exercice
Co-dir. de thèse :	Massimo TISI	Fonction et établissement d'exercice *(si pertinent)*
Co-dir. de thèse :	Théo LE CALVAR	Fonction et établissement d'exercice *(si pertinent)*

Invité·e·(s) :

Prénom NOM	Fonction et établissement d'exercice

ACKNOWLEDGEMENT

Table of Contents

Introduction **9**

1 Model Driven Engineering **11**

1.1 The Unified Modeling Language . . . 11

1.1.1 Meta-Object Facility . . . 12

1.1.2 Class Diagrams . . . 12

1.1.3 Object Diagrams . . . 14

1.2 The Object Constraint Language . . . 15

1.2.1 OCL by example . . . 15

1.2.2 OCL Operations summary . . . 17

1.2.3 Typing OCL Expressions . . . 17

1.2.4 OCL Expression Metamodel . . . 19

1.2.5 OCL Queries and Constraints . . . 21

1.3 EMF and ATL: State-of-the-Art . . . 21

1.3.1 Eclipse Modeling Framework . . . 21

1.3.2 Atlas Transformation Language . . . 23

2 Constrain Programming **25**

2.1 Constraint Satisfaction Problems . . . 25

2.2 Propagation . . . 26

2.2.1 Local Consistency . . . 26

2.2.2 Generalized Arc Consistency and Bound Consistency . . . 26

2.2.3 DPLL and Simplex as Propagators . . . 27

2.3 Global Constraints . . . 27

2.3.1 Global Constraints used in this thesis . . . 28

2.3.2 Propagation of Element . . . 29

2.4 State-of-the-Art . . . 29

2.4.1 Minizinc . . . 29

2.4.2 Choco . . . 31

2.4.3 SWI-prolog and ECL^iPS_e . . . 31
2.4.4 SAT4j and Cassowary . . . 32

3 Model Search using Constraint Programming 33
3.1 Model Search . . . 33
3.1.1 Model Search . . . 33
3.1.2 Applications of Model Search . . . 34
3.2 State of the Art . . . 35
3.2.1 ATL^c . . . 35
3.2.2 EMFtoCSP . . . 37
3.2.3 Alloy & Kodkod . . . 38
3.3 Problem Statement . . . 41

4 Contribution : Object Oriented Constraint Programming 43

5 Contribution : OCL Variable Declaration `VarOperationExpression` 47
5.1 Problem . . . 47
5.1.1 Reconfigurable Manufacturing Systems . . . 47
5.2 Denoting Variables . . . 49
5.2.1 Base Syntax . . . 49
5.2.2 Parameters to guide modeling and and search . . . 50
5.2.3 Vocabulary for Annotated Expressions . . . 51
5.3 Refactoring OCL around annotations . . . 52
5.3.1 Annotation in the OCL Abstract Syntax Tree . . . 52
5.3.2 Refactoring OCL Around Annotations . . . 52
5.4 Experimentation . . . 54
5.4.1 Refactoring Expressions on our method . . . 54
5.4.2 Refactoring Expressions in Alloy Models . . . 55
5.5 Discussions . . . 57

6 Contribution : UML CSP 61
6.1 Encoding Properties . . . 62
6.1.1 Additional *enumerated* encoding for References . . . 63
6.2 Constraint Models for UML Reference Types . . . 64
6.3 Constraint Models for UML Collection Types . . . 64
6.4 Using information from Variable Annotations . . . 65

7 Contribution : OCL Navigation **67**
7.1 CP model for OCL queries on the instance 68
7.2 NavCSP experimentation 70
7.3 Additional navigation model using occurrence variables (bag navigation) . 73
7.3.1 Closure 74

8 Contribution : OCL CSP **75**
8.1 CP Models for OCL Integer and Boolean Operations 77
8.1.1 3-valued logic for OCL 77
8.1.2 Property encoding to 3-valued logic for optional Booleans 78
8.1.3 Operations with Optional Integers 78
8.2 CP Models for OCL Collection to Int Operations 80
8.2.1 size() 80
8.2.2 sum() 80
8.2.3 count(e) 81
8.2.4 max() 81
8.2.5 min() 82
8.3 CP Models for OCL Collection to Bool Operations 83
8.3.1 equals and different 83
8.3.2 includes(e) 85
8.3.3 excludes(e) 85
8.3.4 includesAll(e) 86
8.3.5 excludesAll(e) 86
8.3.6 isEmpty() 87
8.3.7 notEmpty() 87
8.3.8 forall(exp) 88
8.3.9 exists(exp) 89
8.3.10 one(exp) 89
8.4 CP Models for OCL Collection Operations 90
8.4.1 select(exp) 90
8.4.2 reject(exp) 90
8.4.3 any() 91
8.4.4 sortedBy(exp) 91
8.5 CP Models for OCL Collection Type Casting Operations 92

8.5.1 asBag() 92
8.5.2 asSet() 93
8.5.3 asSequence() 93
8.5.4 asOrderedSet() 96
8.5.5 Filtering Dummy Values in OCL Collection Operations 97
8.6 CP Models for OCL Sequence and Ordered Set Operations 98
8.6.1 src.prepend(e) 98
8.6.2 src.append(e) 98
8.6.3 src.insertAt(i,e) 99
8.6.4 src.subSequence(s,f) 100
8.6.5 src.at(p) 101
8.6.6 src.indexOf(p) 101
8.6.7 src.first() 101
8.6.8 src.last() 101
8.6.9 src.reverse() 102
8.7 CP Models for OCL Set and Bag Operations 103
8.7.1 union(exp) 104
8.7.2 Intersection 105
8.7.3 unordered - unordered 106
8.7.4 symmetricDifference(exp) 106
8.8 CP Models for OCL Bag Operations 108

Conclusion **109**
Première section de l'intro 110

Bibliography **113**

Introduction

The Object Constraint Language (OCL)[1] is a popular language in Model-Driven Engineering (MDE) to define constraints on models and metamodels. OCL invariants are commonly used to express and validate model correctness. For instance, logical solvers have been leveraged to validate UML models against OCL constraints, used by tools like Viatra [**?**], EMF2CSP [**?**] and Alloy [**?**]. However several problems in MDE require a way to automatically enforce constraints on models that do not satisfy them, e.g. to complete such models or repair them. Because of its combinatorial nature, the problem of enforcing constraints can be computationally hard even for small models.

Constraint Programming (CP) aims to efficiently prune a solution space by providing tailored algorithms. Such algorithms are made available in constraint solvers like Choco [**?**] in the form of global constraints. Leveraging such global constraints would potentially increase the performance of constraint enforcement on models. However, mapping OCL constraints to global constraints is not trivial. Previous work [**?**] has started to bridge from arithmetic OCL constraints to arithmetic CP models, but it exclusively focused on constraints over attributes.

In this paper we focus instead on structural constraints, i.e. OCL constraints that predicate on the links between model elements. In detail, we present a method in two steps: 1) we provide an in-language solution for users to denote CP variables in OCL constraints; 2) we describe a general CP pattern for enforcing annotated structural OCL constraints, i.e. constraints predicating on navigation chains. To evaluate the effectiveness of the method, we discuss the size of the Constraint Satisfaction Problems (CSPs) it produces, and the resolution time in some examples.

In the context of Model-Driven Engineering (MDE), models represent structured data, and the model of the data structure is known as a metamodel. The Unified Modeling Language (UML) [2] provides visual languages, such as class and object diagrams, to define both models and meta-models. The Object Constraint Language (OCL) [3] complements UML by enabling the specification of constraints over models, based on the underlying

[1] `https://www.omg.org/spec/OCL/2.4/`
[2] `https://www.omg.org/spec/UML/2.4`
[3] `https://www.omg.org/spec/OCL/2.4`

metamodel concepts. The Eclipse Modeling Framework (EMF) [4] supports UML and OCL, enabling validation of models against their meta-models and associated constraints. It also includes model transformation tools such as ATL [**?**, **?**], an OCL-based language that expresses mappings between meta-models. ATLc [**?**] extends ATL by introducing model space exploration capabilities to facilitate transformation specification. It leverages constraint solvers to generate and visualize model instances, which users can then adjust or repair using solver feedback. The primary use of ATLc is to create a Graphical User Interface for a model, allowing the user to easily edit the model. This generally breaks some of the user defined OCL constraints, and our work hopes to provide a way to repair the models around the user's choices. The core problem is: given a metamodel, a partial model and model constraints as input, the objective is to find model instances that satisfy the metamodel and model constraints. ATLc does so by interpreting part of their OCL expressions upon an instance as a constraint satisfaction problem (CSP), which can be solved by engines like Cassowary (for linear programming) or Choco (for constraint programming). However, ATLc is currently limited to single-valued model attributes, using integers or reals. Our work seeks to generalize this approach to support collection-valued properties: attributes and relations.

Among existing tools, Alloy [**?**] stands out as a tool offering a dedicated language for defining meta-models and constraints. Alloy is often used for specification testing–such as verifying security protocols or code–by searching for models that satisfy given constraints. It can also be used for checking specifications by searching for valid instances or counterexamples. Alloy has also been applied to model transformation and model repair [**?**], with some approaches translating UML/OCL into Alloy specifications [**?**,**?**]. The core difference with our approach lies in the underlying solving technique: Alloy is based on SAT solving, while we use Constraint Programming (CP). Choosing between SAT and CP for model search tasks is not straightforward, and through our experimentation, we aim to shed some light on that choice in the context of model search. Related work leveraging CP, global constraints and similar models also exists [**?**], however UML/OCL coverage doesn't include the general case of collection properties discussed in this paper, and required for the experimentation.

[4] `https://projects.eclipse.org/projects/modeling.emf.emf`

Chapter 1

Model Driven Engineering

Model Driven Engineering (MDE) has emerged as a central paradigm in software development. It's history is intertwined with that of Object Oriented Programming (OOP). Where the OOP paradigm can be summarized as *everything is an object*, MDE is similarly described as the paradigm where *everything is a model*. Two of the longest standing actors are the Object Management Group (OMG) which notably provides specifications for both the Unified Modeling Language and the Object Constraint Language, and Eclipse providing the Eclipse Modeling Framework (EMF). EMF allows for the generation and testing of code using UML specifications, the design and use domain specific languages, and the manipulation of data by means of model transformation. More recent efforts include JetBrains' Meta Programming System (MPS) which focuses on providing tools to develop and use Domain Specific Languages.

Models are used because they are: cheaper, safer, easier to manipulate and learn from. For software development, software engineers build models of the desired software. Such models allow the engineers to quickly iterate and test their design, but also serve as a language to communicate with the software developers who will implement the software. Beyond software development, models are similarly used: scaled-down models of buildings are easier to make and iterate upon and can be used to communicate with stake holders, weather models provide forecasts which are crucial to farming, digital-twins allow for the monitoring and interaction with complex systems.

In the following sections we'll present UML and OCL. We'll then present the state-of-the-art for MDE, and the tools we are building upon.

1.1 The Unified Modeling Language

UML provides a range of languages for modeling purposes. We need languages to describe metamodels and models. For models, UML provides Object Diagrams, and for metamodels, Class Diagrams and the Object Constraint Language. Additionally, we will use C-like

syntax for Class specifications.

1.1.1 Meta-Object Facility

The Unified Modeling languguage is build ontop of the Meta-Object Facility standard. Formally: the UML metamodel conforms to MOF meta-metamodel.

The MOF standard also describes a four layer model hierarchy illustrating its application. The MOF meta-metamodel exists as the top layer of abstraction in the MOF architecture, the M3 layer. The MOF architecture has three lower layers of abstraction, the lowest being that of the system under study, the M0 layer. The UML metamodel exists at the M2 layer, and the M1 layer contains the UML conforming models and metamodels created by the user to represent the system under study. The UML languages we'll use to specify metamodels and models will are outlined in the following sections on class diagrams, object diagrams and the object constraint language.

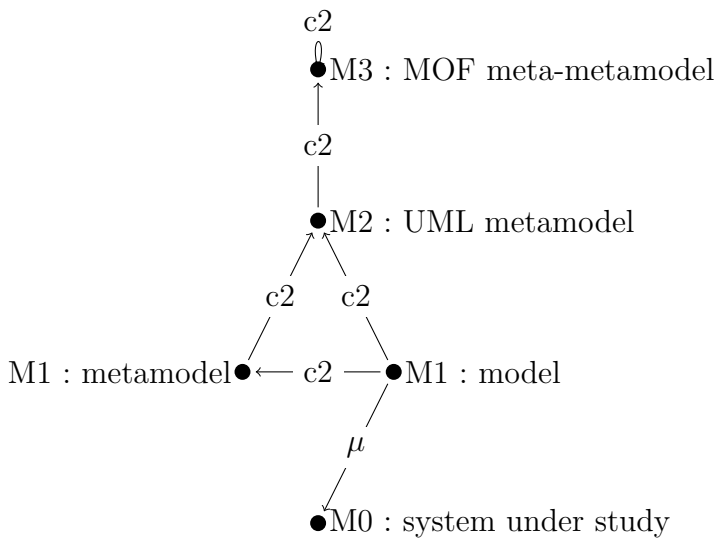


Figure 1.1: Meta-Object Facility model hierarchy

1.1.2 Class Diagrams

Class Diagrams identify concepts and their properties. In a family tree for instance, the core concept is Person, with attributes such as age and references such as parent (or its opposite, child) to express relationships between people.

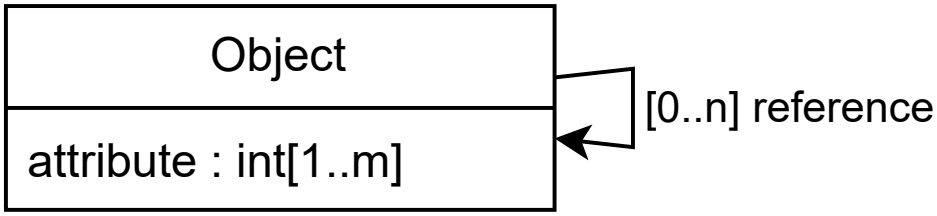


Figure 1.2: UML Class Diagram as Metamodel

Here we present a generic metamodel. It describes a class named `Object`, which has two properties: `attribute`: a collection of integers, with at least one and at most m elements, `reference`: a collection of up to n references to other `Object` instances. These illustrate the two main types of properties in object-oriented modeling: Attributes, which store intrinsic data values (e.g., numbers or strings), References, which define relationships between objects in the model.

```
class Person {
    age : int,
    parents : Person[2..2] set
}
```

Here we have a C-like or Alloy-like description of the `Person` metamodel. We will commonly use `:` to express type, and `[]` for collections.

UML Collection Types

UML allows properties to be collections, and distinguishes four standard collection types, based on two dimensions: order and uniqueness.

- `Sequence`: ordered, allows duplicates – e,g., [2,3,1,1],
- `Bag`: unordered, allows duplicates – e.g., [1,1,2,3],
- `Set`: unordered, unique elements only – e.g., [1,2,3],
- `OrderedSet`: ordered, unique elements – e.g., [2,3,1].

An important note is that ordered doesn't pertain to the values. In [2,3,1,1]: 2 is the first value, and 1 is the last value. The intended collection type can be indicated in the class diagram using annotations such as `ordered`, `unique`, or `seq` (for sequences).

UML Reference Types

UML also gives us different kinds of relations between objects, notably containment relations and and opposite relations.
Opposite The opposite of *child-of* for instance, is *parent-of*. Sometimes references may need to be one-way for encapsulation.
Containment means that the *contained* object is only related to one *container* object through this relation.

1.1.3 Object Diagrams

An object diagram describe instances of the concepts defined in a class diagram. For example, Figure 1.3 shows an instance conforming to the class diagram in Figure 7.1. It includes three objects, each identified by a unique ID (e.g., `o1`, `o2`, `o3`). For instance, object `o1` has as attribute a collection of 3 integers and is connected to other objects (e.g., `o2` and `o3`).

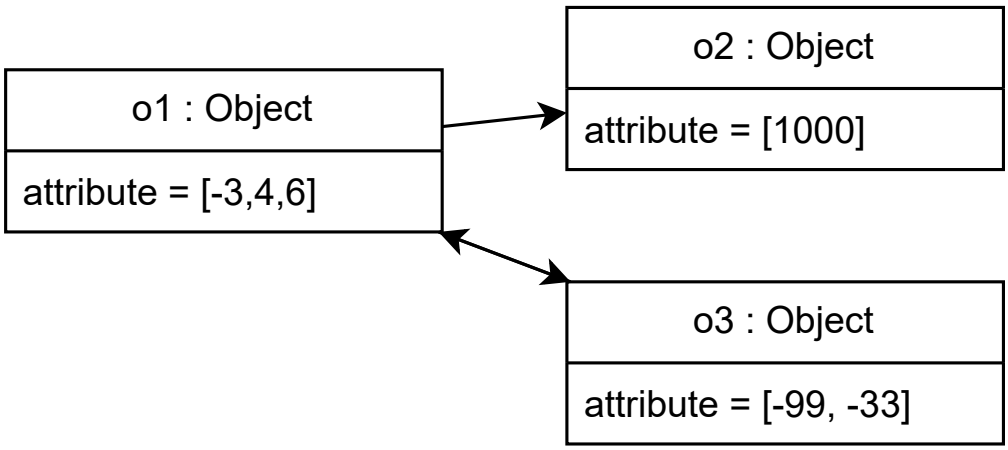


Figure 1.3: UML Instance Diagram as Model

Here each rectangle represents an object, and the title area is used to give the `:Type` and optionally a name. Below the title area, we again list the attributes and the information associated. The arrows between the rectangles represent instances of the references between objects. In Figure 1.3 we see a model which conforms to the metamodel from Figure 7.1, for which we chose $n = 2$ and $m = 3$. This here shows objects of the type

described by the class, and the values assigned to their properties. In this model, `o2 : Object` is an instance of the `Object` class named `o2`, as attribute it has a collection of integers with a single value: 1000. The object `o1` has as attribute a collection of 3 integers and is connected to both other objects, which are the maximums allowed by our choice of $n = 2$ and $m = 3$.

1.2 The Object Constraint Language

The Object Constraint Language (OCL) is a declarative language used to specify additional rules and constraints on UML models that cannot be expressed using diagrams alone. It enables the formalization of conditions that instances of the model must satisfy, serving as a powerful complement to class and object diagrams.

Given an instance such as the one shown in Figure 1.3, OCL is typically used to verify whether it satisfies the specified constraints. In this work, however, we aim to use OCL as a means to guide model search, thereby enabling the completion or correction of partial or inconsistent data. To this end, we propose an approach that reformulates OCL specifications as constraint satisfaction problems (CSPs). This paper focuses on how OCL's collection typing, defined in the Class Diagram, and type casting operations can be modeled using global constraints over bounded domains.

1.2.1 OCL by example

OCL is designed to be easy to read an write. So for an initial introduction we'll use a simple example, and identify the core parts of the language. The primary use of OCL is to add additional constraints to a class diagram.

Figure 1.4: Simple class diagram describing people

This class diagram describes people, giving their age, and listing their children.

However, a constraint such as "a child must be younger than their parents" cannot be represented directly in a class diagram. However, it can be expressed in OCL as follows:

```
context Person inv:
  self.children.age.forall(a| a < self.age)
```

This constraint states that for every Person instance, all of their children must be younger. The `context` keyword specifies the class to which the constraint applies, and `inv` stands for invariant, i.e., a condition that must always hold true. This invariant states that for every Person, the age of each parent must be greater than the person's age.

self: identidying objects in the model

The expression `self` refers to the current object of the set identified by the context. It identified the main variable of the expression. `self.attribute` returns its attribute values, and `self.reference` retrieve related objects. Chained queries like `self.reference.attribute` retrieve the attributes of referenced objects.

Building OCL expressions by chaining operations

OCL expressions a built using function composition.

$$source.operation(arguments)$$

OCL operations generally are applied OCL expressions identified as *source*, which can be understood as the first argument for the operation. Operation arguments generally take the form of OCL expressions.

Navigation: relation to the metamodel

The primary operation on object models, is accessing their properties, and navigating the graph of objects. OCL provides language to navigate the model and retrieve it's information.

Property Access OCL uses a `.property` notation for property access.

For example to access the age and the children of a person, in ocl you would write.

`self.age`

```
self.children
```

Navigation

To access the children's age, we need to navigate away from the object designated `self`, to the objects representing their parents, and access their age attribute.

In the general case, if the source expression results in a collection of objects, we can navigate their references to a new collection of objects and access their properties.

```
source.prop
```

1.2.2 OCL Operations summary

OCL supports a rich set of operations on primitive types and collections. Examples include: Boolean expressions (`forall`, `exists`, `not`, `and`, `or`), Arithmetic and comparison (`+`, `-`, `>`, `<`), and Collection operations (`sum`, `size`, `includes`, `asSet`, `asSequence`, etc). Each collection type comes with its own operations and can be explicitly cast using operations like `asSet()`.

This cheat sheet outlines all the available OCL expressions, organized by the types the can be applied to.

1.2.3 Typing OCL Expressions

Typing is the process of assigning a data type to expressions, variables, and operations in a language. It ensures that operations are performed on compatible data types, preventing logical errors and enhancing code clarity. In the context of Object Constraint Language (OCL), typing plays a critical role in defining constraints and queries over UML models, ensuring that expressions are both syntactically and semantically valid. OCL is statically typed, meaning types are checked at compile-time rather than runtime.

This diagram specifies the type system for OCL.

`InvalidType` represents incorrect OCL expressions. `VoidType` represents *absent* values. Such as when `self.children` points to no one. `Class` represents the use of class names from the metamodel in the model constraint expressions. `DataType` represents the data that can be found in the model, inferred from it, and the literals in the ocl expressions. `PrimitiveType` represents the type of data that can be used, such as: integers, reals, booleans, strings. `CollectionType` represents collections of data.

Object Constraint Language (OCL) Cheat Sheet eScribis

OCL SYNTAX

cs	e op e	ns:: ... ns::id (7.5.7)
id	e . id (7.4.10)	**if** pd **then** e **else** e **endif**
self	e . pt (e, ... , e) (7.4.10)	**let** id = e : T, id2 = e:T, ... **in** e2 (7.4.3
	c -> pt (e, ..., e) (7.4.10)	

OCL LIBRARY

Type	Examples	Operations
Integer *(11.5.2)*	1, -5, 34	i+i2, i-i2, i*i2, i.div(i2), /, i.mod(i), i.abs(), i.max(i2), i.min(i2), <, >, <=, >=, i.toString()
Real *(11.5.1)*	1.5, 1.34, ...	r+r2, r-r2, r*r2, r/r2, r.floor, r.round(), r.max(r2), r.min(r2), <, >, <=, >=, r.toString()
Boolean *(11.5.4)*	true, false	not b, b and b2, b or b2, b xor b2, b implies b2, b.toString()
String *(11.5.3)*	'', 'a chair'	+, s.size(), s.concat(s2), s.substring(i1,i2), s.toInteger(), s.toReal(), s.toUpperCase(), s.toLowerCase(), s.indexOf(s2), s.equalsIgnoreCase(s2), s.at(i), s.characters(), s.toBoolean(), <, >, <=, >=
Enumeration *(7.4.2)*	Day::monday, Day::tuesday, ...	=, <>
TupleType(**x : T1,** **y : T2,** **z : T3)** *(7.5.15)*	Tuple { y = 12 x = true, z:Real= 3.5 }	t.x t.y t.z
Collection(T) *(11.7.1)*		=, <>, c->size(), c->includes(o), c->excludes(o), c->count(o), c->includesAll(c2) c->excludesAll(c2), c->isEmpty(), c->notEmpty(), c->max(), c->min(), c->sum(), c->product(c2), c->selectByKind(ty), c->selectByType(ty), c->asSet(), c->asOrderedSet(), c->asSequence(), c->asBag(), c->flatten(), *(11.9.1)* c->any(it|pd), c->closure(it|e), c->collect(it|e), c->collectNested(it|e), c->exists(it1,it2...|pd), c->forAll(it1,it2...|pd), c->isUnique(it|e), c->one(it|pd), c->reject(it|pd), c->select(it|pd), c->sortedBy(it|e), c->iterate(e|)
Set(T) *(11.7.2)*	Set {1,5,10,3}, Set{}	st->union(st2), st->union(bg), st->intersection(st2), st->intersection(bg) st - st2, st->including(e), st->excluding(e), st->symmetricDifference(st2)
Bag(T) *(11.7.4)*	Bag {1,5,5} Bag {}	bg->union(bg2), bg->union(st), bg->intersection(bg2), bg->intersection(st) bg->including(e), bg->excluding(e)
OrderedSet(T) *(11.7.13)*	OrderedSet{10,4,3} OrderedSet{}	os->append(e), os->prepend(e), os->insertAt(e), os->subOrderedSet(i1,i2), os->at(i), os->indexOf(e), os->first(), os->last(), os->reverse()
Sequence(T) *(11.7.4)*	Sequence{5,3,5} Sequence{}	sq->union(sq2), sq->append(e), sq->prepend(e), sq->insertAt(i,o) sq->subSequence(i1,i2), sq->at(i), sq->indexOf(o), sq->first(), sq->last(), sq->including(e), sq->excluding(e), sq->reverse()
Class		cl.allInstances()
Global functions		e.oclIsTypeOf(ty), e.oclIsKindOf(ty), e.oclAsType(ty) e.oclIsInState(state), e.oclIsNew()

i : Integer	c : Collection(T)	os : OrderedSet(T)	cs: constant	ty : type
r : Real	st: Set(T)	t : Tuple(…)	pd : predicat	it : iterator
b : Boolean	bg : Bag(T)	id: identificateur	e : expression	cl : classifier
s : String	sq : Sequence(T)	pt: property	ns: namespace	

Figure 1.5: snippet from the OCL Cheat Sheet provided by eScribis

To color our previous ocl expression on the Person metamodel:

$$self.children.forall(c|c.age < self.age)$$

`.children` is of type `CollectionType` `.age` resolves to an integer which is a `PrimiteType`. `forall` and `<` resolve to a boolean which is a `PrimitiveType`.

This is also a decent metric for OCL coverage. In this thesis we will be mainly focusing expressions which resolve to a CollectionType and the Integer and Boolean PrimitiveType. We will also have some cases of Class type, notable when handling `Person` in the expression `Person.allInstances()` We will also have to manage InvalidType and VoidType.

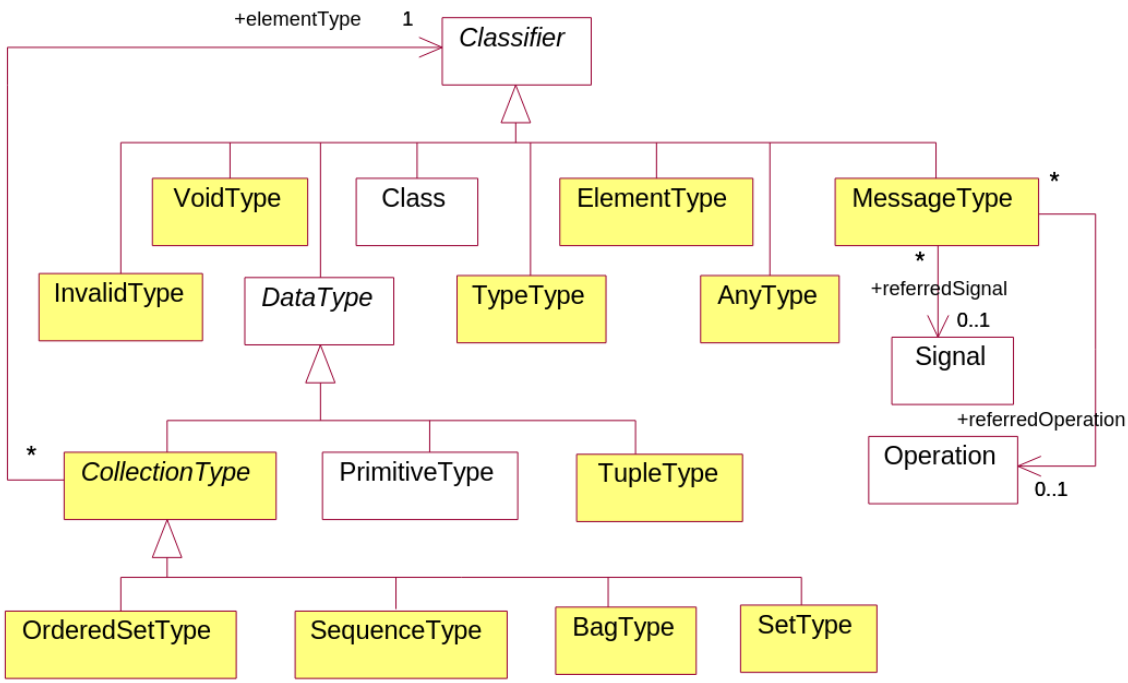


Figure 1.6: OCL Types

There are also limitations with the standard OCL typing system, especially around invalid and void expressions, and efforts such as OCL propose a typing system based on relational logic. Similar to Alloy. In relational logic all expressions are resolve to sets. Invalid and Void types are essentially encoded as the empty set .

→ **MC** we leverage the type system to define CSP / type is checked before building csp

1.2.4 OCL Expression Metamodel

OCL being a domain specific language in the model driven ecosystem, we naturally have a class diagram modeling the language. → **MC** explain The OCL metamodel describes the "kinds of words" which appear in the OCL language, and the patterns in which they appear.

The top level concept is that of an OCL expression: `OclExpression`. As previously stated, all OCL expressions are typed, hence they all inherit from the concept of `TypedElement`.

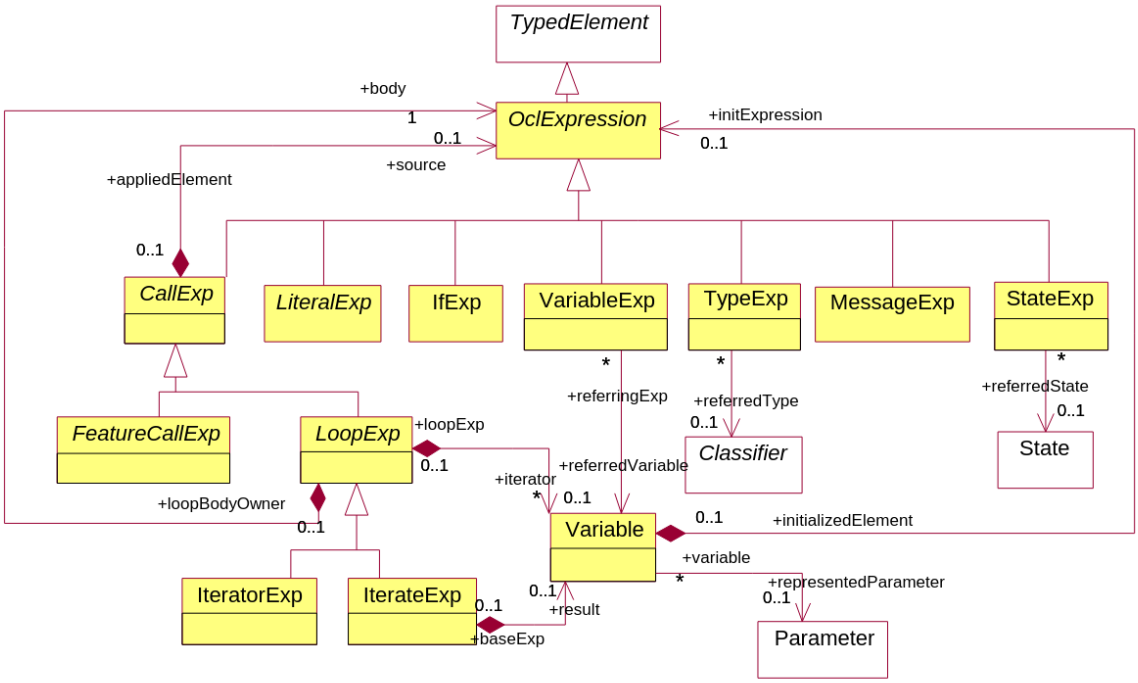


Figure 1.7: OCL Expression Metamodel (simplified)

To color our previous ocl expression on the Person metamodel:

$$self.children.forall(c|c.age < self.age)$$

`self` and `c` are `VariableExp`. `.children` and `.age` are `FeatureCallExp`. `forall` is a `LoopExp`. `<` is a `CallExp`.

The CallExp represents most of the words in ocl, notably model navigation and operation calls. Generally an CallExp has a source `OclExpression` to which it is applied. The simplest operations represented by `CallExp` are those applied to primitives, such as arithmetic operations for integers, and logical operations on booleans and integers: $+, <, \wedge$. The `FeatureCallExp` represents property access, providing attribute values and the objects resulting from references. Naturally the type of what is returned from property access varies greatly.

The `LoopExp` groups operations applied to expressions of type `Collection`, notably iterate, and its special cases such as: sum, `size`, forall, etc. . . which make up the majority of the collection operations. The `LoopExp` generally has a `body:OclExpression`, except for builtins such as `sum`. If a `LoopExp` has a body, it requires an `interator:Variable`.

The expressions of type `TypeExp`, resolve to OCL types and Classes defined in the metamodel. Notably used in expressions such as `Person.allInstances()`, or `src.select(e|e.oclisType` in which `Person` is a `TypeExp`.

The `LiteralExp` allows users to write literal expressions such as `3` or Sequence(2,1,3,1). The `IfExp` exists for compatibly with older version and can be replaced by `implies`.

The expressions of type `MessageExp` and `StateExp` are not covered in this thesis.

1.2.5 OCL Queries and Constraints

In this thesis we often categorize OCL expressions as either *queries* or *constraints*.
Query Expression: `self.ref.select(r | predicate(r)).prop.sum()` OCL expression resulting in a collection of integers or objects.
Constraint Expression: `inv: self.ref.forall(r | p(r))` OCL expression resulting in a boolean. Constaint expressions can be *enforced* or *reified*. In this invariant the *forall expression* `self.ref.forall(...)` is enforced, and the *predicate* `p` can be said to be a reified constraint.

1.3 EMF and ATL: State-of-the-Art

1.3.1 Eclipse Modeling Framework

The Eclipse Modeling Framework is a suite of tools for the Eclipse ecosystem, allowing users design data structures, manipulate structured data and generate code.

In this screenshot we can EMF being used to create a Class Diagram to design a zoo application. From this metamodel, we'll be able to generate a tool to edit object models representing instances, and serialize them in XMI files.
XMI XML Metadata Interchange, a file format specified by OMG for the serialization of MOF Models. EMF provides tools to load and save models from and to XMI files. Loading an XMI model into and EMF editor generally requires the Ecore metamodel it conforms to, and generates EObjects which can be manipulated. In this listing we see a simple model, with two linked objects, each object holds some information.
Ecore is the EMF format for class models (metamodels). Ecore is an XML-like language based on XMI. Loading an Ecore file entails instantiating EObjects of type EClasses, EAttributes and EReferences corresponding to the classes of the class model and their

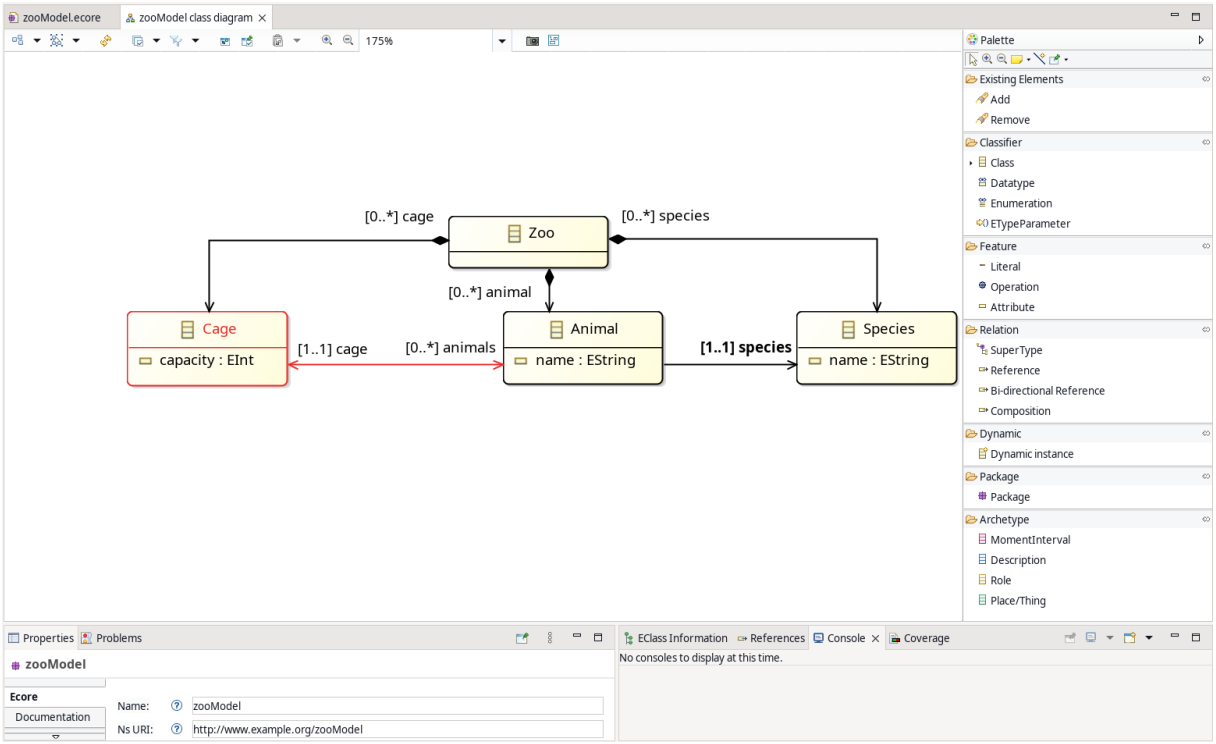


Figure 1.8: Using the Eclipse Modeling Framework to edit an Ecore metamodel using a Class Diagram editing tool

```
<Model>
    <object att="information␣one" ref="//@object.1"/>
    <object att="information␣two" ref="//@object.0"/>
</Model>
```

Listing 1: Minimal Object Model in the XMI format

properties, and generating code to manipulate them and their corresponding objects. EMF provides tools so visualize these metamodels as Class Diagrams.

TODO: Add Ecore metamodel.

EObject java interface representing objects. Provides access to the EClass it conforms to. Provides access to the information via *getters* and *setters*, and the EReference or EAtrribute representing the class property.

EClass java interface representing classes. EClasses provide access to the properties.

EReference java interface representing properties of type reference. Provides access to the name, type, and collection cardinality.

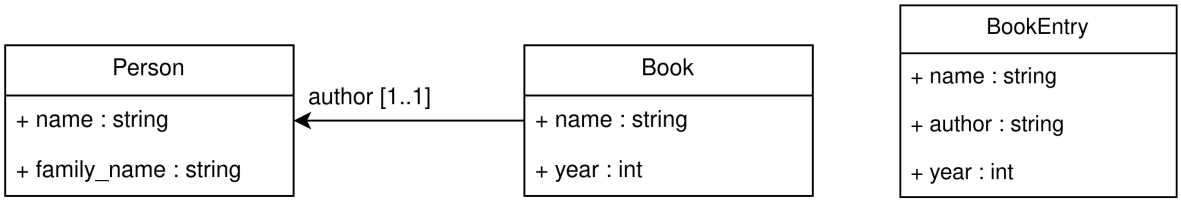
EAttribute java interface representing properties of type attribute. Provides access to

the name, type, and collection cardinality.

1.3.2 Atlas Transformation Language

The Atlas Transformation Language (ATL) provides a framework for specifying and executing model-to-model transformations within the Eclipse Modeling Framework. ATL is designed to automate the conversion of models from one metamodel to another, enabling the creation of model transformation pipelines—a critical feature for modern Model-Driven Engineering workflows. ATL extends OCL to define transformation rules that operate across multiple classes. These rules establish bindings, which associate properties between source and target model elements using OCL queries. This allows ATL to express complex mappings between metamodels in a declarative style, focusing on what should be transformed rather than how. These rules associate properties across the classes using OCL queries, in what is called: a binding.

Just like OCL, ATL can be written declaratively, but ATL also allows for imperative sections, which are sometimes necessary to specify *steps* required to transform an object into another.



((a)) Source metamodel: people and books

((b)) Target metamodel: library book entry

In these two figures we find a source and a target metamodels. The source metamodel describes books and their relation to people. And the target metamodel is a specification for a book entry in a library.

```
// ATL transformation rule
rule BookToBookEntry {
from
    b : BookMM!Book
to
    be : LibraryMM!BookEntry (
        title <- b.title ,
```

```
            authorName <- b.author.family_name.toUpperCase() + b.author.
                name,
            year <- b.year
        )
    }
}
```

In this example, we focus on a transformation rule, which produces `BookEntry` objects from `Book` objects and their associated `Person` objects. A rule has two parts: a `from` section which identifies source classes and `to` which identify target classes, and binds their properties to those of the source. In the binding `authorName <- b.author.name + b.author.first_name`, we link the `authorName` property of an entry `be`, to the conjunction of the `name` and `family_name` of the `Person` identified by the `author` property of the book `b`.

Chapter 2

Constrain Programming

Constraint Programming [**?**] is a powerful paradigm that offers a generic and modular approach to modeling and solving combinatorial problems. In the wider field of artificial intelligence, constraint programming stands out by expressing problems in terms of variables, domains, and constraints. This declarative approach allows for reusable models and leverages advanced constraint propagation and search techniques to efficiently explore solution spaces.

2.1 Constraint Satisfaction Problems

A CP model consists of a set of variables $X = \{x_1, \ldots, x_n\}$, a set of domains $\mathcal{D}$ mapping each variable $x_i \in X$ to a finite set of possible values $dom(x_i)$, and a set of constraints $\mathcal{C}$ on X, where each constraint $c(X(c), R)$ defines a set of values that a subset of variables $X(c)$ can take.

Domains can be either bounded, defined as an interval $\{lb..ub\}$, or enumerated, explicitly listing all possible values (e.g., $1, 10, 100, 1000$). This distinction impacts the choice of constraints: for instance, the global cardinality constraint is more effective with enumerated domains. An assignment on a set $Y \subseteq X$ of variables is a mapping from variables in Y to values in their domains. A solution is an assignment on X satisfying all constraints.

Constraint programming also provides for other types of discrete domains for variables, such as Set and Graph variables. Sets domains often use enumerations as lower and upper bound, to represent what *must* be in the set, and what *might* be in the set. Graph domains generally employ set variables to represent links from a node in a graph. For models of which the variables domains are boolean or real are covered in fields distinct from constraint programming, such as *Boolean Satifiability*, *Satifiability Modulo Theories* and *Linear Programming*.

2.2 Propagation

Propagation is one of the fundamental pillars of constraint programming, along with modeling and search. Global constraints spanning a large number of variables allows one to leverage propagation to the fullest. The application of propagation to the problem of OCL is our fundamental difference to much of the related work. To apply it to OCL we need a systematic way to model OCL expressions using gobal constraints, and particularly to model OCL query expressions.

2.2.1 Local Consistency

→ **MC** Local Consistency is around a single constraint, enforced/checked by its propagation algorithm

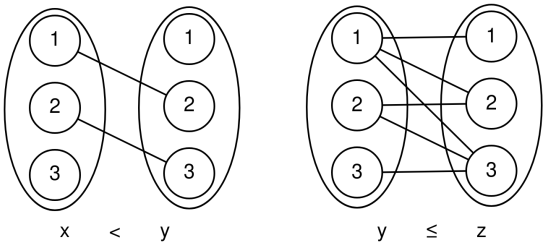


Figure 2.1: Illustration of Generalized Arc Consistency and Bound Consistency on $x < y \leq z$

In this figure we see local consistency enforced over the expressions $X < Y$ and $Y \leq Z$. For every possible value of each domain, we can find at least one value in the other domain(s) which satisfy the constraint.

2.2.2 Generalized Arc Consistency and Bound Consistency

→ **MC** General Arc Consistency is ideal, Bound Consistency is reasonable → **MC** These generalized consistencies can be approched by combining local consistencies In this figure we illustrate *generalized arc consistency* (GAC) above and *bound consistency* (BC) below, for the CSP: $x < y \leq z$. To illustrate the advantage of bound consistency we extended the domain from [1..3] to [1..300], making it 100 times bigger while keeping the same encoding. For generalized arc consistency, we can see the enumerated domains of X, Y and Z. Across the domains of the variables, we link possible valuations satisfying the

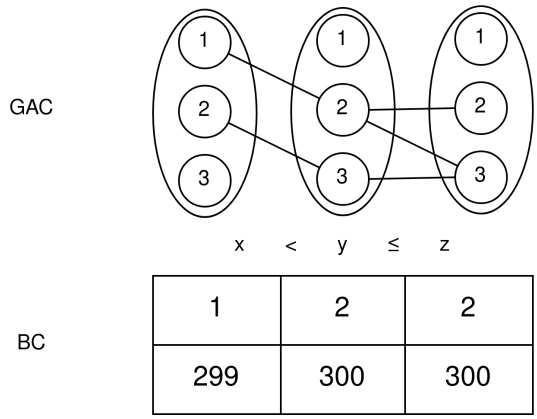


Figure 2.2: Illustration of Generalized Arc Consistency and Bound Consistency on $x < y \leq z$

whole problem: for instance if $X = 1$, we are connected to two valid assignments for Y and Z. Bound consistency on the other hand, only requires an encoding of the upper and lower bounds of the domain, for which it maintains consistency in the same way as GAC.

2.2.3 DPLL and Simplex as Propagators

There are notable classes of CSP which come with their own specialized algorithm to solve them. When the variable domains are all 0/1 and the constraints in Conjunctive Normal Form (CNF), the DPLL algorithm is the best propagator. Similarly when the variable domains are all Real numbers, and the constraints are a system of linear equations, the Simplex algorithm is the best propagator. Traditionally, SAT and linear programming are distinct fields of symbolic AI, each with their own class for solvers. However from the context of constraint programming, we can understand them global constraints and propagators, and CP solvers can employ these specialized solvers.

2.3 Global Constraints

Global constraints provide shorthand to often-used combinatorial substructures. More precisely, a global constraint is a constraint that captures a relationship between several variables [**?**, **?**], for which an efficient filtering algorithm is proposed to prune the search tree. In other words, the “global" qualification of the constraint is due to the efficiency of its filtering algorithm, and its capacity to filter any value that is not globally consistent

relative to the constraint in question. Global constraints are thus a key component to solving complex problems efficiently with CP.

2.3.1 Global Constraints used in this thesis

In this thesis we limit ourselves to the global constraints provided by the Choco Solver. Some notable examples of global constraints used in this thesis are:

- Element is useful when "selecting a variable from a list" is part of the problem. Let $X = [x_1, \ldots, x_n]$ be an array of integer variables, $z \in \{1, \ldots, n\}$ be an integer variable representing the index, and y be an integer variable representing the selected value. $element(y, X, z)$ [**?**,**?**] holds iff $y = x_z$ and $1 \leq z \leq n$, this means that variable y is constrained to take the value of the z-th element of array X. This constraint is also known as *nth* in systems like ECL^iPS_e and swi-prolog.

- Regular expression constraints are very expressive when describing sequences of variables, and offers powerful filtering. Let $X = [x_1, \ldots, x_n]$ be an array of integer variables and A be a finite automaton. $regular(X, A)$ [**?**] enforces that the sequence of values in X must form a valid word in the language recognized by the automaton A.

- The $stable_keysort(X, Y, z)$ [**?**,**?**] defined over two matrices of integer variables holds iff (1) there exists a permutation π s.t. each row y_k of Y is equal to the row $x_{\pi(k)}$ of X ($k \in \{1, \ldots, i\}$); (2) the sequence of rows in Y, truncated to the first z columns, is lexicographically non-decreasing; (3) if two rows in X have equal key values for the first z columns, then their relative order in Y must match their original order in X. Table **??** illustrates with an instance that satisfied this constraint.

- Cumulative is generally used for scheduling tasks defined by their start time, duration and resource usage: $< s_i, d_i, r_i >$. It requires that at any instant t of the schedule, the summation of the amount of resource r of the tasks that overlap t, does not exceed the upper limit C. The values of t range from: a the earliest possible start time s_i, to b the latest possible end time $s_j + d_j$. Let $S = [s_1, \ldots, s_n]$ be the start times of n tasks, $D = [d_1, \ldots, d_n]$ their durations, $R = [r_1, \ldots, r_n]$ their resource demands, and C the total capacity of the resource (a constant). $cumulative(S, D, R, C)$ [**?**, **?**] holds iff $\forall t \in [a, b], \sum_{i|s_i \leq t < s_i + d_i} r_i \leq R$ [**?**]. where $a = min(s_0, .., s_n)$ and $b = max(s_0 + d_0, .., s_n + d_n)$,

2.3.2 Propagation of Element

Propagation for a constraint is the action of updating the domains of the variables bound by that constraint. When solving, propagations will generally run when the domain of one of the variables bound by the constraint is updated.

For instance, let $y = \{0, 1\}, x_0 = \{0\}, x_1 = \{2, 5\}, z = \{-10..10\}$ be the domains of the variables given to the element constraint. The element constraint's propagator can update the domain of z to $\{0, 2, 5\}$. The meaning of this propagation is, the possible values for z are a subset of the union of possibilities for x_y, here the union of x_0 and x_1. If during another constraint's propagation, or during search, 0 is removed from the domain of z, such that $z = \{2, 5\}$, the element constraint can update the domain of y to just $\{1\}$. Here, because the domains of x_0 and z are disjoint, then z can not be equal to x_0, hence the element constraint propagation can remove 0 from the domain of y. Finally, if the element constraint is given the following variable instances: $y = 0, x_0 = 0, z = 2$, propagation for the constraint would tell us it is not satisfiable, and serve as a counter proof in model validation.

2.4 State-of-the-Art

CP solvers use backtracking search to explore the search space of partial assignments. The main concept used to speed up the search is constraint propagation by *filtering algorithms*. At each assignment, constraint filtering algorithms prune the search space by enforcing local consistency properties like *domain consistency* (a.k.a., *Generalized Arc Consistency* (GAC)). A constraint c on $X(c)$ is domain consistent, if and only if, for every $x_i \in X(c)$ and every $v \in dom(x_i)$, there is an assignment satisfying c such that $(x_i = v)$.

To use a solver, you can specify a model using a modeling language such as prolog, minizinc and DIMACS. Many solvers also provide an API for a general purpose programming language such as Java.

2.4.1 Minizinc

MiniZinc is a high-level, declarative constraint modeling language. It allows users to model complex combinatorial optimization and decision problems in a solver-independent way. The language is designed to be readable and expressive, focusing on what needs to be solved rather than how to solve it. FlatZinc is a low-level, solver-specific intermediate

language. When you run a MiniZinc model, it is first compiled into FlatZinc, which can then be executed by any solver that supports the FlatZinc format.

```
include "globals.mzn";          % for alldifferent
int: N = 16; %size of the sudoku
int: M = 4; %sqrt of M
int: MM = 3; %M-1 for subgrid logic
array[1..N,1..N] of var 1..N: X;

% Numbers in Rows and Columns are all different
constraint forall(i in 1..N)(
    let{ array[int] of var int: row = [ X[i,j] | j in 1..N ],
         array[int] of var int: col = [ X[j,i] | j in 1..N ]} in
    alldifferent(row)
/\ alldifferent(col)
);

% Numbers in Groups are all different (3x3 groups in a 9x9 sudoku)
constraint forall(i,j in 0..MM)(
    let{ array[int] of var int: grp =
        [ X[ii+i*M,jj+j*M] | ii,jj in 1..M ]
    } in
    alldifferent(grp)
);
```

Listing 2: Sudoku model in Minizinc

This model encodes a Sudoku puzzle of size N×N (here, N=16, so a 16×16 grid) using constraint programming. The globals.mzn library is included for the alldifferent global constraint.

This specification starts with the declaration of the constants, and the variables and their domains. Among the constants are: N the size of the Sudoku grid (16). M Square root of N (4), defining the size of each subgrid. MM equal to M-1 (3), used for indexing subgrids. `X[1..N,1..N]` is a 2D array of decision variables, each representing a cell in the Sudoku grid. Each cell can take a value from 1 to N.

Then we have a specification of the constraints. We first have the constraints for rows and columns. For every row and column, all values must be different (alldifferent). The we have the constraint for subgrids. The grid is divided into M×M subgrids (here, M=4, so 4×4 subgrids). All values in each subgrid must also be different.

2.4.2 Choco

Choco is the primary solver used for our implementation. It is also used by ATL^c, Grimm, and by the QVTc approach to organize ECL^iPS_e prolog rules. Choco Solver is an open-source Java library for constraint programming. It allows users to model and solve complex combinatorial problems by defining variables, domains, and constraints with language similar to a minizinc specification.

Choco is among a class of solvers which provide implementations for the various global constraints. It is known for its rich set of global constraints, such as cumulative for scheduling, and globals for graph variables. Coupled with advanced search strategies, Choco a powerful tool for both research and industry, widely used in scheduling, planning, optimization, and decision-making applications. Additionally Choco supports the FlatZinc format, and internally employs a SAT solver for a *CNF* global constraint and can interface with linear programming solvers.

2.4.3 SWI-prolog and ECL^iPS_e

SWI-Prolog and ECLiPSe Prolog are both powerful Prolog systems, each with a distinct focus but sharing core strengths in logic and constraint programming. Both provide support for many global constraints, enabling efficient modeling and solving of complex combinatorial problems. At their core, they combine Prolog's logical inference and search strategies, with constraint propagation: as constraints are posted, propagation algorithms reduce variable domains and detect inconsistencies, while Prolog's backtracking search explores possible solutions, resulting in a hybrid approach that leverages the strengths of both paradigms.

SWI-Prolog is a general-purpose, open-source Prolog environment, renowned for its extensive libraries, and active community. It integrates constraint logic programming, making it versatile for a wide range of applications—from knowledge representation and natural language processing to constraint-based problem solving.

ECLiPSe Prolog is specialized for constraint logic programming, offering built-in, advanced constraint solvers (like finite domain and linear constraints) out of the box. It is particularly suited for complex combinatorial problems, scheduling, and optimization, and is widely used in both research and industry for these purposes.

2.4.4 SAT4j and Cassowary

Both these solvers from fields distinct from constraint programming, namely Boolean satisfiability problems and linear programming.

SAT4J is a Java library for solving Boolean satisfiability problems, where constraints are expressed in conjunctive normal form (CNF) over 0/1 variables. It efficiently handles propositional logic constraints, making it suitable for problems reducible to SAT, such as configuration, verification, and combinatorial search. SAT4J is often used as a back-end for constraint programming systems when dealing with discrete, logical constraints.

Cassowary is a simplex-based constraint solver designed for systems of linear equations and inequalities over real numbers. It is widely used for layout and graphical constraint solving, such as UI design and geometric problems, where constraints are expressed as linear relationships between real-valued variables. Cassowary's incremental solving and efficient handling of under-constrained systems make it ideal for interactive applications.

Chapter 3

Model Search using Constraint Programming

3.1 Model Search

Model search exists at the interesection of Model Driven Engineering and Artificial Intelligence. EMF has a need for tools assisting in the modeling process. Model search is a powerful tool with many applications. One of the open questions in MDE is *how to effectively find models that conform to a metamodel*. When the metamodel has model constraints the problem may become complex.

Because MDE leverages formal languages for modeling purposes a natural choice of AI are those solving Constraint Satisfaction Problems.

3.1.1 Model Search

Model search is aptly defined in → **MC** cite Kleiner. It is defined as a transformation towards a model conforming to a metamodel, and starting from a partial model which conforms to a relaxation of the metamodel.

The transformation is described with the following megamodel, which is not too dissimilar to that of ATL.

Relaxed Metamodel A relaxed metamodel is a metamodel for which a subset of the constraints are not enforced. These constraints include property cardinalities and model constraints.

Partial Model A partial model conforms to a relaxed metamodel, and partially conforms to the metamodel which was relaxed. This generally means it is populated with Objects, but missing information on the values of referrences and attributes. For a `Person`, this means not having their age, name, or know which other people they are related to. Another example would be generating a unique number for the person, which satisfies a set of

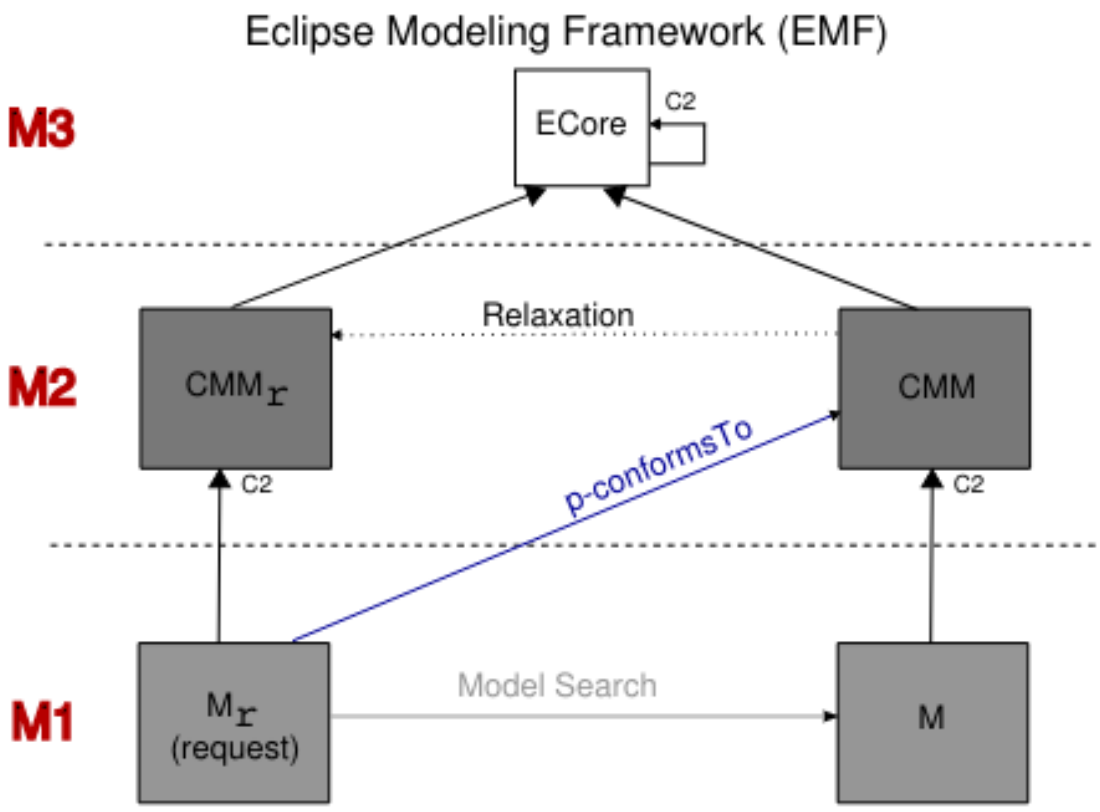


Figure 3.1: Model Search

constraints.

Partially-conforms-to The relation between a Partial Model and the metamodel which was relaxed.

3.1.2 Applications of Model Search

Model Instantiation is the process of generating a model that conforms to a metamodel. Generally this means providing a partial model, with all the object instances. The count of objects per class is often a parameter for model instanciation. This is particularly useful for validating metamodels.

Model Completion is the process of taking a model with missing information and inferring possible information. Generally this means providing a partial model, with all the object instances and some immutable data for the objects. The missing data is the

reason the model only partially conforms.
Model optimization is a transformation towards an optimal model with respect to an objective function.
Model Repair is the process of making a conforming model from a non-conforming model.
model space exploration: classical model transformation tools associate one source model to one target model. Model space exploration involves transformations transformations towards sets of models. Domain Space Exploration describes a system of model transformations, model space exploration uses transformations over sets to model the system of model transformations.

3.2 State of the Art

3.2.1 ATL^c

ATL^c provides an extension to the ATL allowing users to add OCL invariants to the targets of transformation specifications. ATL^c then translates the conjunction of these invariants into a CSP which can be solved by a range of solvers, such as: Choco and Cassowary (Simplex). In ATL^c invariants, only integer attributes can be variables, and variables are identified by the last `PropertyCallExp` of a query expression.

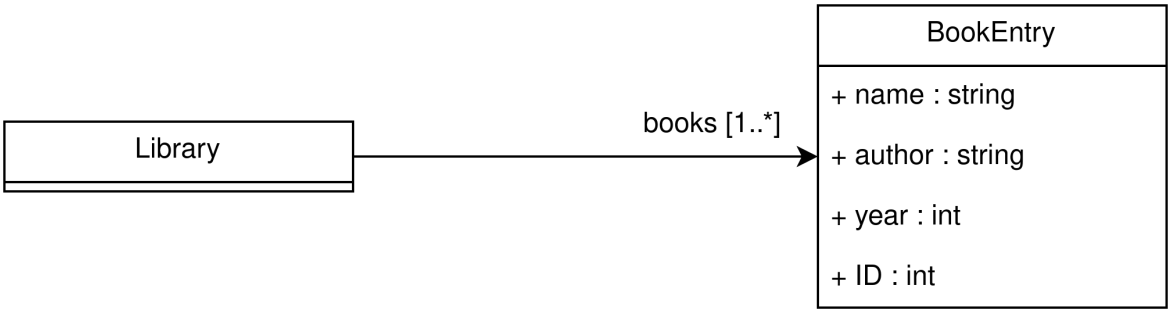


Figure 3.2: Target metamodel: Library

```
rule BookToBookEntry {
from
    b : BookMM!Book
to
    be : LibraryMM!BookEntry (
```

```
6                title <- b.title,
7                authorName <- b.author.family_name.toUpperCase() + b.author.
                     name,
8                year <- b.year
9            )
10           cstr : Constraints!Constraint(
11               value <- (be.ID / 1000) = year
12           )
13       }
14   }
15       rule CreateLibrary {
16       to
17           lib : LibraryMM!Library (
18               books <- BookMM!Book.allInstances()->collect(b | thisModule.
                     BookToBookEntry(b))
19           )
20           cstr : Constraints!Constraint(
21               value <- lib.books.ID->isUnique()
22           )
23   }
```

Here we see how to add ATL^c invariants to a familiar ATL transformation, which constrain the ID number for the book entries. For the library rule the `to` section, we see two targets: a `Library` object, for which the reference `books` is populated by a standard binding, and a `Constraint` object. The `cstr` target allows the user to add invariants to a transformation rule. Here the invariant states that *books in a library must have unique IDs*. This invariant replaces a specification of how to make a unique ID. ATL^c assumes `ID` is the variable in this invariant, not the `books` relation, as `ID` is the last `PropertyCallExp` in the query `lib.books.ID`.

Describing a satisfactory model in such away, can be easier and shorter than describing how to make a satisfactory model. A notable example is positioning GUI elements in the provided JavaFX metamodel. This kind of problem is best solved with a solver such as Cassowary. By chaining ATL^c model transformations on can also chain solvers best suited to each aspect of the problem. In this figure, we see an example result from a chain of ATL^c transformations. A first transformation repairs Schedule edits using Choco, and the second transformation generate an interactive visualization using Cassowary to position the elements.

However ATL^c doesn't cover all of OCL; we can't solve the problem where the relation

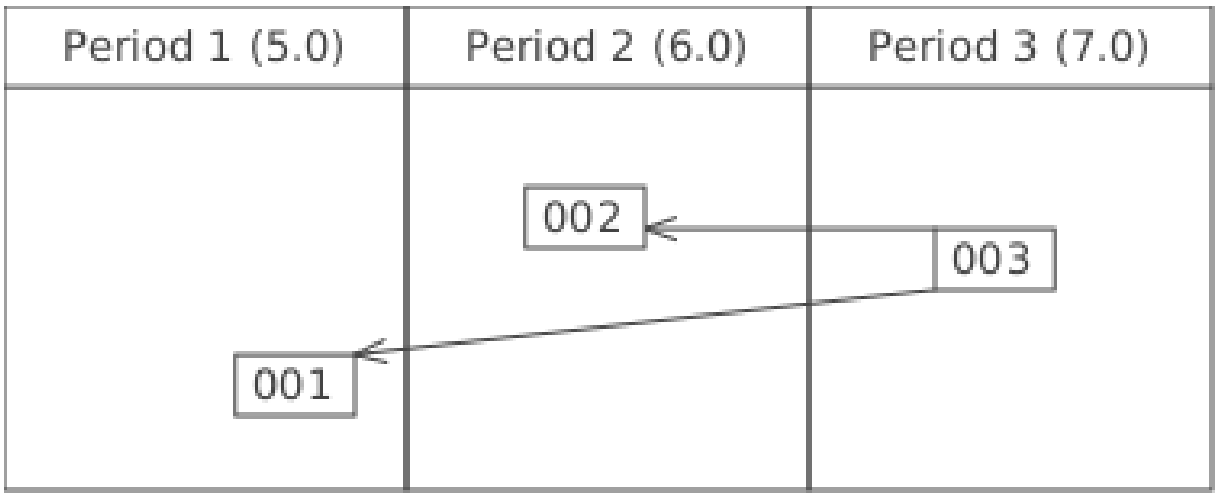


Figure 3.3: Scheduling App resulting from a chain of ATL^c model transformations

between libraries and books is missing. To do so, first requires being able to identify variables in invariants to continue leveraging the standard OCL interpreter to find the variables. And second: requires further coverage of UML and OCL, for the translation of OCL expressions to constraint models.

In the context of Model Search, the standard ATL transformation provides the relaxed-metamodel and produces the partial model. The relaxed-metamodel is completed with the ATL^c invariants, and a solver is leveraged to finish the transformation.

3.2.2 EMFtoCSP

→ **MC** Prolog is a natural way to implement a language

→ **MC** SWI-prlog and ECL^iPS_e also provide constraint programming techniques ontop of prolog techniques

→ **MC** EMF2CSP uses ECL^iPS_e to neatly describe OCL, and leverage gobal constraints such as *element* (like us)

→ **MC** Important note (un-like us): This method will generate a different problem for each combination of collection sizes. Whereas we add the "what size is the collection" problem to the CSP.

→ **MC** related work OCL#

3.2.3 Alloy & Kodkod

Alloy provides a framework for modeling problems and solving them using *off-the-shelf* SAT solvers. The alloy language is inspired by the Z modeling language and the Object Constraint Language. It provides both a means to specifiy class models and model constraints.

Alloy Language

```
sig BookEntry {
    year : Int,
    ID : Int
}{
    ID.div[10] = year
}

sig Library {
    books : set BookEntry
} {
    #books=#books.ID
}
```

Listing 3: Minimal Object Model in the XMI-like format

In this listing we see our Library Example. In alloy a class can be modeled using signatures. We can see, signatures are similarly named lists of properties. Properties are similarly typed using primitives for attributes, and other signatures for references. Properties in alloy hold a single value by default, but can declared to be a collection, such as set and sequence. Signatures can also be used as the context for model constraints. Such as here *the number of books, must equal the number of IDs* This expressions accesses the ID property of the library's books, using notation similar to OCL.

Kodkod

Kodkod is an API to model relational first order logic problems and translate them to Conjunctive Normal Form (CNF) model. Alloy uses this API to translate models for solving with SAT solvers.

Here we have a simple alloy specification which simply illustrates how types and relations are modeled when looking at the resulting *relational first order logic* specification.

```
sig S {R : one T}
sig T {}
run {#S=2 #T=2}
```

Listing 4: Simple Alloy model, with two related concepts, with two instances each

This specification also describes the partial model, by stating there should be two objects of type `S`, and two of type `T`

$$
\begin{aligned}
\{a_1, a_2, a_3, a_4\} \\
S :_1 [\{\langle a_1 \rangle, \langle a_2 \rangle\}, \{\langle a_1 \rangle, \langle a_2 \rangle\}] \\
T :_1 [\{\langle a_3 \rangle, \langle a_4 \rangle\}, \{\langle a_3 \rangle, \langle a_4 \rangle\}] \\
R :_2 [\{\}, \{\langle a_1, a_3 \rangle, \langle a_1, a_4 \rangle, \langle a_2, a_3 \rangle, \langle a_2, a_4 \rangle\}] \\
\forall s \in S : |R.s| = 1
\end{aligned}
\tag{3.1}
$$

Here we have a *relational first order logic* model of the alloy specification. The first line lists the atoms. There is one atom for each object of the model. When working with integers, each possible int value implies an additional atom. The next two lines represent the unary relations *is of type S* and *is of type T*. These relations are encoded as a set, for which we have the lower bound (objects that must be in the set) and upper bound (objects that maybe in the set). For these two relations `S` and `T` have identical upper and lower bounds, meaning its a constant of this problem. They identify atoms a_1 and a_2 as being of type S, and a_3 and a_4 of type T.

Next is the binary relation `R`, which describes the links from objects of type S to those of type T. If we look at the upper bound for the possibilities we see → **MC** stuff like $\langle a_1, a_3 \rangle$, meaning maybe a_1 is linked to a_3. We have no information about existing relations between S and T objects, so the lower bound is empty.

The final line uses relational language such as `R.s`: *R join s*, to predicate on the relations. The s in $\forall s \in S$, will be instantiated as singletons relations, which can be joined with the `R` relation, to get the subset of `R` corresponding to the s object. Ultimately the alloy expression `one T` gets translated to $|R.s| = 1$, or the size of the subset resulting from the join $R.s$ is equal to one.

To include integers in such a model implies having atoms for each value of the integer domains. This has an important impact of the size of integers users can model with. For instance in the *Book ID starts with year* constraint, requires a large domain to span a

few centuries. With the partial model from this listing, linking a library to three books,

```
//1134759 vars. 49163 primary vars. 4416342 clauses. 14524ms
//Instance found. Predicate is consistent. 246893ms. //4min
one sig A extends BookEntry{}{year=356}
one sig B extends BookEntry{}{year=356}
one sig C extends BookEntry{}{year=365}
one sig L extends Library {}{books = A+B+C}

run {
    #BookEntry=3
    #Library=1
} for 13 Int
```

Listing 5: Partial model and resolution parameters

of which 2 some from a same year. The dates are offset, to better fit in the bit-width, and there are 10 possible values for the last digit in the reference number. However solving this problem took minutes.

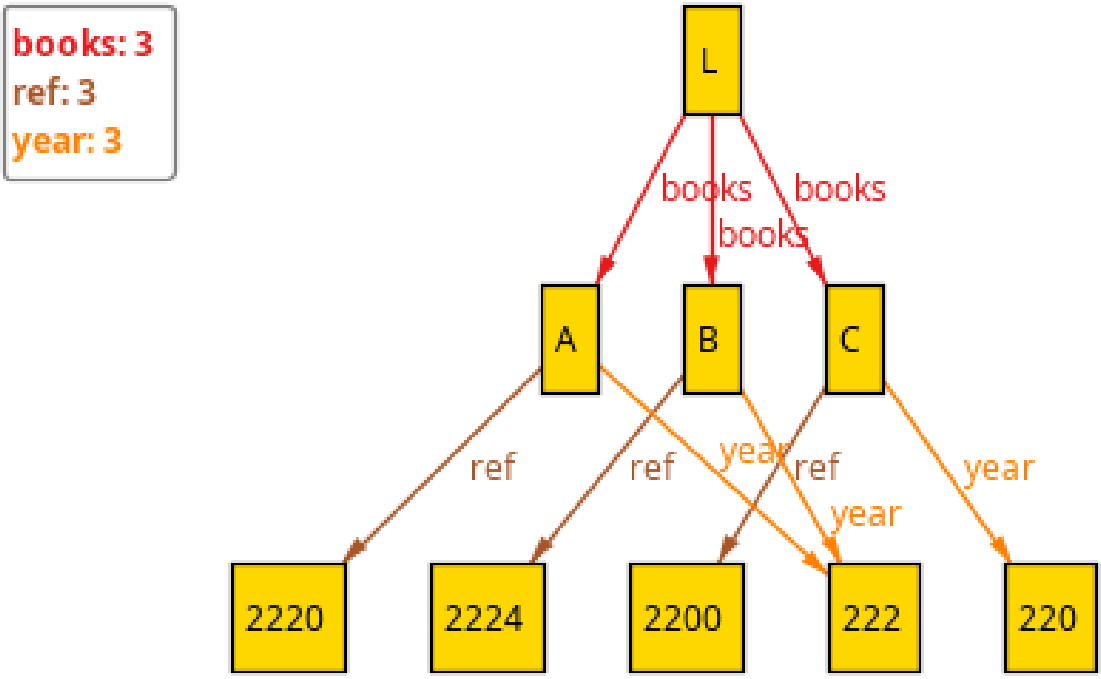


Figure 3.4: Alloy visualization of the solution

This solution illustrates each possible integer being an object in the model. Each object here corresponds to an atom. Unused integers are hidden in this diagram, but they do

need exist in the relational first order logic model as atoms.

If you want to have a sequences, we need to add atoms for integers. And the relation `R` from the relational first order logic model becomes a ternary relation where the tuples now include the integer atom representing the position of the link in the sequence: $\langle a_6, a_1, a_3 \rangle$ which would mean the first (1=a_6) link between from a_1:S is to a_3:T. Which further increases complexity in alloy models.

→ **MC** Similar to this work: OCLsolve

3.3 Problem Statement

→ **MC** Model Search is an important problem → **MC** EMF provides an industry standanrd for models → **MC** EMFtoCSP aptly uses prolog to describe the language, and the prolog flavour provides global constraints but backtracking on collection sizes don't seem great, and tool is hard to get working because old → **MC** Alloy Kodkod provide a very powerful framework for solving relational problems on sets, but it struggles with large integers → **MC** ATL^{c}an interesting framework to build upon, but has an incomplete coverage of OCL

→ **MC** We will continue in the spirit of ATL^c, builing a tool it can employ but can also independtly can be used for Model Search with UML and OCL. We will use variables modeling integers, encode for sequences first, add the collection size problem to the CSP, use the Best Models^{TM} and try to approach Bound Consistency across the whole model.

Chapter 4

Contribution : Object Oriented Constraint Programming

In this chapter we will give an overview of our method for building a CSP from an EMF instance with OCL constraints.

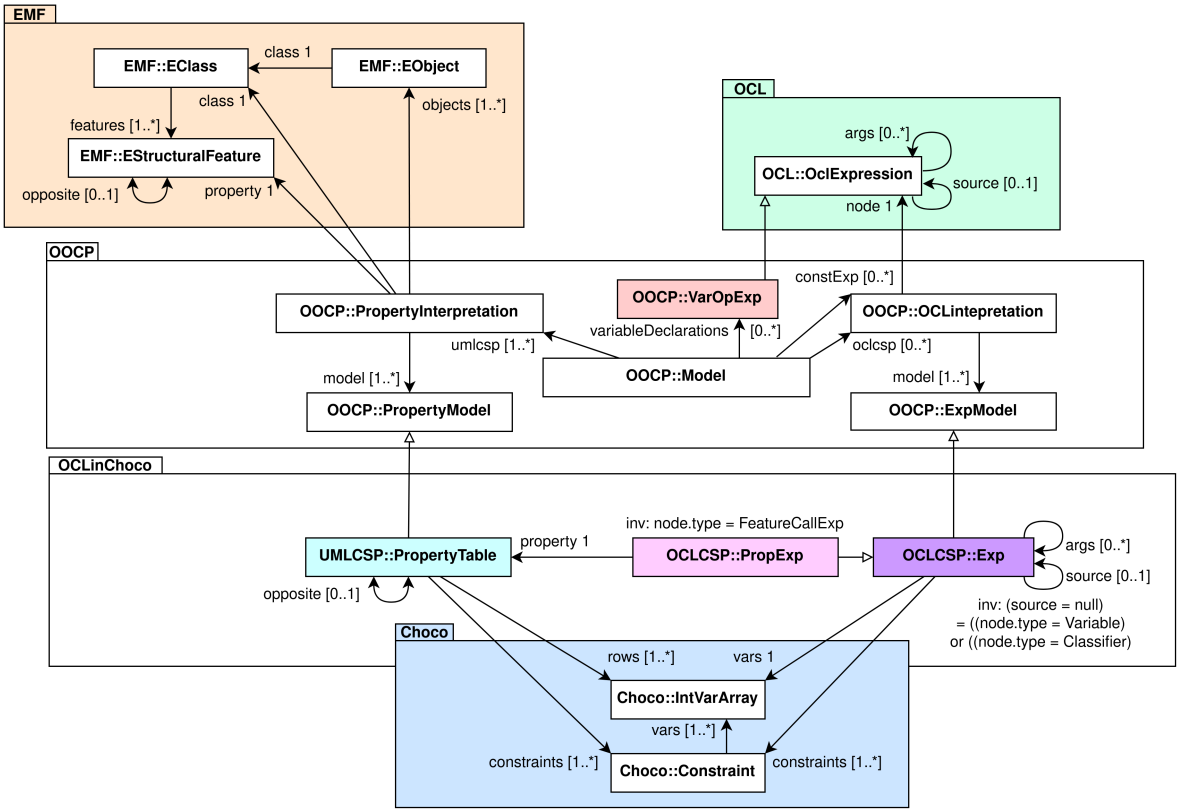


Figure 4.1: architecture of OOCP models

The interpreter is broken up into two main projects *OOCP*, which builds constraint satisfaction problems from EMF instances and OCL constraints, and *OCLinChoco* which models the semantics of UML and OCL using Choco. Not present in this diagram are

the concepts such as *solver* and *strategy*, as is focuses on the CSP. The EMF package is coloured orange, from which we'll be accessing the concepts of `EObject` modeling the instance, `EClass` which describe the objects, and `EStructuralFeature` which describe the objects' properties. These are described in chapter 1.3. In green we have the OCL package, described in chapter 1.2, from which we have its most general concept `OclExpression`. Finally at the bottom, in blue, we have the Choco package, providing our variables, constraints, and the solver. These are all described in chapter 2.

The central package is *Object Oriented Constraint Programming* (OOCP), which runs the interpretation associating EMF model elements and constraints, to interconnecting parts of a constraint satisfaction problem. An interpretation can also employ several models and/or solvers. It is also responsible for handling the `VarOpExp` expression, in red. The use of `VarOpExp` to declare variables, identifies parts of the instance to model, and allows OOCP to dispatch OCL interpretation between a standard interpretation for *constant expressions*, and the CSPs interpretation for *variable expressions*.

The package making up the majority of this thesis is *OCLinChoco*, for which we'll have three parts: the UML CSP in light blue, the navigation CSP in pink, which is a special part of the OCL CSP in purple. For the chapters describing OCLinChoco, we

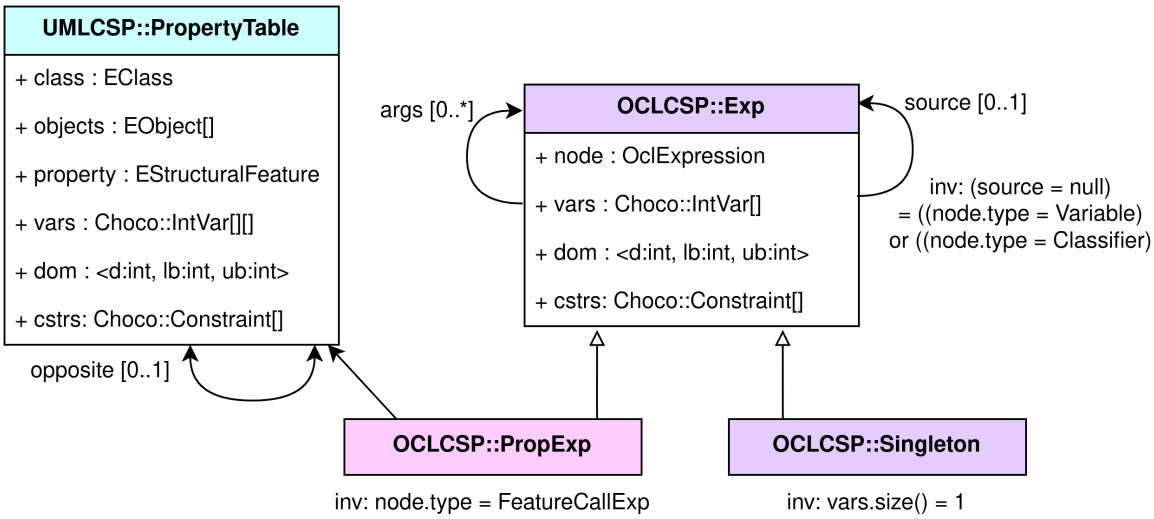


Figure 4.2: OCLinChoco metamodel

have a simplified metamodel which focuses on those sections. This model outlines how each the UML CSP and OCL CSP associate EMF, OCL and Choco concepts, and how UML CSP, OCL CSP and navCSP concepts are related. We will reuse this diagram for

each of those chapters to show *where we are*, and will further explain each part of this metamodel in the relevant chapters.

Chapter 5

Contribution : OCL Variable Declaration `VarOperationExpression`

5.1 Problem

For illustrative purposes, throughout the paper we exemplify the method on a well-known example, about Reconfigurable Manufacturing Systems (RMS). Notice however that the way to enforce constraints presented in this paper can be applied to any class diagram with OCL constraints.

5.1.1 Reconfigurable Manufacturing Systems

An RMS [**?**] is an industrial solution to the problem of varying product demand. In the most common version of the problem (from [**?**]), a factory is organised into subsequent stages of identical machines. To change the productivity of the factory, machines are added and removed from stages. Manufacturing tasks are allocated to stages, and are generally at least partially ordered. RMSs provide a number of problems to solve, such as: matching tasks and machines with stages, optimising those matches to achieve new productivity goals, as well as allocating the products to a machine of the stage it's going through, or planning machine maintenance.

A Class Diagram for RMS

Figure 5.1 uses a class diagram to describe the concepts of an RMS, and how they relate (inspired by [**?**]). In this figure, we focus on the graph structure of the model (classes and references among them), omitting attributes. We use the class diagram flavor from the Eclipse Modeling Framework (EMF)[1], that connects classes by unidirectional references, instead of bidirectional associations.

[1] `https://eclipse.dev/modeling/emf/`

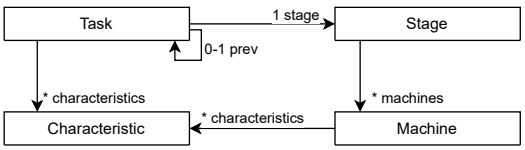


Figure 5.1: Class Diagram for RMS Task constraints
[class diagram of RMS]

```
1 context Task inv SameCharacteristicConstraint:
2     self.stage.machines.forall(m | m.characteristics
3         ->includesAll(self.characteristics))
4
5 context Task inv PrecedenceConstraint:
6     self.stage.stageNum >= self.prev.stage.stageNum
```

Listing 6: RMS Task constraints in OCL.

The two main components of a reconfigurable manufacturing system are stages, and machines which are organised into stages. A `Machine`'s property is its relation to a set of `Characteristics`. Objects of type `Task` are partially ordered, as expressed by the `prev` reference. Tasks have two other properties: a reference to a `Stage` (allocating the task to that stage), and a reference to characteristics (i.e., the machine characteristics needed to perform the task). Similarly to the example in [**?**], tasks and machines can be linked to any number of characteristics.

OCL Constraints for RMS

The class diagram shown in Figure 5.1 cannot encode all constraints that are required for an instance to be a correct RMS instance. Additional constraints can be specified using OCL.

6 shows the two constraints we use as running example in this paper. These are the structural constraints for tasks, part of a more detailed constraint model for RMS, with budget and productivity constraints.

OCL provides a way to query a model. For a given object, or collection of objects, we can query properties using `"."`, in expressions following an *objects.property* pattern. Their properties can be references or attributes. In our running example, tasks have a reference to a stage, named `stage`, representing the stage a task is assigned to. In the OCL in 6 line 2, from the context of a `Task`, we query its `stage` by the expression `self.stage`. The sub-expression `self` resolves to an individual object of type `Task`. The whole expression

resolves to another object, of type `Stage`, associated with the task. We can see this as a navigation starting at a `Task` node, and traversing the reference to the associated `Stage` node. We can chain navigations: e.g. to find the machines of the stage of a given task in 6 line 2 we use `self.stage.machines`.

`SameCharacteristicConstraint` ensures that the machines of the stage performing a task have all the required characteristics. From the given task (`self`) we navigate to the stage where it is performed, and find all the machines of that stage (`self.stage.machines`). For each machine `m` we impose that the set of `characteristics` it supports (`m.characteristics`) includes all the characteristics required by the given task (`self.characteristics`).

`PrecedenceConstraint` ensures that tasks with precedence are performed after their predecessors, i.e. they are assigned to the same stage or a later one.[2]

We will consider these constraints in the following three scenarios.

S1: machines have already been assigned to stages, and the assignment of tasks to stages must be found;

S2: tasks have already been assigned to stages, and the assignment of machines to stages needs to be found;

S3: both tasks and machines must be assigned to stages.

5.2 Denoting Variables

5.2.1 Base Syntax

To allow users to explicitly denote properties (attributes or references) in an OCL expression as variables (variable attributes or variable references), we propose the `var()` operator with the following syntax:

```
source.var('property')
```

where `source` identifies the objects resulting from the prior sub-expression, `property` is the name of one of the attributes or references of the objects.

[2]The constraint uses the `stageNum` constant integer attribute that indicates the position of the stage in the manufacturing line, omitted in Figure 5.1.

In **??** we apply the operator to `SameCharacteristicConstraint` in 6 for each one of our three scenarios, defining the properties that we consider as variables for that scenario. All properties that are not included in a `var()` operation call are considered constant.

Notice that our in-language solution does not extend the syntax of the OCL language, but we add an operation to the OCL library: `var(propertyName: string) : OclAny`. When the OCL constraint is simply checked over a given model (and not enforced), the `var` operation simply returns the value of the named property (as a reflective navigation).[3] Whereas, if one wants to enforce OCL, `var` is used as a hint to build the corresponding CSP.

5.2.2 Parameters to guide modeling and and search

We can add extra parameters to `var` to drive CP modeling, e.g. for bounding the domain for a property, or choosing a specific CSP encoding among the ones presented in the next section.

- **max_card** allows a user to set the maximum number of elements in the property. Required if a property's max cardinality is set to infinite.
- **min_card** allows a user to set the minimum number of elements in the property.
- **lb** allows a user to set a lower bound for the domain of the variables encoding the property.
- **ub** allows a user to set an upper bound for the domain of the variables encoding the property.
- **enumerate-domain** applies an encoding which enumerates the domain, allowing for additional filtering models.
- **branch-priority** make the solver prefer to branch off these variables.
- **branch-max** make the solver prefer to try large values for these variables.
- **branch-min** make the solver prefer to try small values for these variables.
- **fill** allows a user to preserve the values that may already be present for the property.

[3] Look at getRefValue from ATL/OCL for a similar reflective operation `https://wiki.eclipse.org/ATL/User_Guide_-_The_ATL_Language#OclAny_operations`

- **stay** guides model repair, uses ATL^c weights for weighted CSP.

- **temp** guides model repair, positive temperatures allow for values to change, negative values allow for values to resist change.

Note that, alternatively, users can also annotate the variable references in the metamodel, instead of the constraints. In this case, we can always statically translate such variable annotations on metamodels into the variable annotations on constraints discussed here.

5.2.3 Vocabulary for Annotated Expressions

Variable Property: these imply adding variables and possibly constraints to the UML CSP. Non-annotated properties will be called constant. From annotation of the OCL we infer which properties are variable. This is the bridge between the model instance and the CSP, when we find a structure in the CSP, we will update these references.

Variable Expression: if an OCL expression has an annotation, it is referred to as variable. If it has none we call it constant. Expressions can be decomposed into sub-expressions, and variable expressions can be decomposed into variable and constant sub-expressions. A particular type of expression is the query, which in this paper are the primary sub-expressions of structural constraints.

Variable Query: variable queries are similarly any annotated query expression, but can be divided into two main parts:

1. variable property access: `src.var(prop)`

2. variable navigation: `.var(src).prop`.

Variable property access is sourced from constant (non annotated) queries, e.g. `self.prev.var(stage`
Variable navigation is sourced from a variable query. For example in: `self.var(stage).var(machine`
machines and characteristics are reached through variable navigations, the first being from `Stage` to `Machines`, the second from `Machines` to `Characteristics` .

5.3 Refactoring OCL around annotations

5.3.1 Annotation in the OCL Abstract Syntax Tree

The annotated OCL is parsed in the form of an AST. Given an instance model to solve for, each object will have their own instance of the AST, where `self` resolves to said object. **??** shows the AST of `SameCharacteristicConstraint` from **??** Scenario S3. We show `var` annotations as dotted rectangles.

?? illustrates a key function of `var` annotations: they define the scope of the CP problem, i.e. a frontier between what can be simply evaluated by a standard OCL evaluator, and what needs to be translated and solved by CP. In **??**, the scope defined by each `var` annotation is indicated by a dotted rounded rectangle. The `var` annotation requires everything inside the corresponding scope to be translated to CP.

For instance, since the reference between `Task` and `Stage` is annotated (`self.var('stage')`), the result of the `stage` `NavigationOrAttributeCallExp` needs to be found by the solver. All nodes in the scope of an annotated node will be in the CSP, as what they resolve to depends on the solution the solver is searching for. Conversely, nodes that are not in the scope of any `var` annotation do not need to be translated to CP, making the CP problem smaller.

The processing of the AST in **??** (corresponding to Scenario S3) starts from the bottom: `self` is directly evaluated by standard OCL, as is `self.characteristics`. However we don't know the result of `self.stage`, which implies we don't know the result of `self.stage.machines`. Above, we iterate on the unknown machines and for all of them: ask what their characteristics are, and if they include the characteristics of the task. All these questions must also be answered by the solver, which means any node of the tree within the dotted box must be resolved by the solver.

In Scenario S2, `SameCharacteristicConstraint` from **??** has the same AST as in **??**, but only the `machines` node is annotated as `var`. Hence, in this case the CP scope is smaller, since `self.stage` can be directly evaluated by OCL.

5.3.2 Refactoring OCL Around Annotations

Given that everything above an annotated node of the AST is within the scope of the CSP, it's interesting to find strategies to reduce the scope as much as possible, as it results in a smaller CSP to solve. The annotated expressions of **??**, all have their annotations low in

```
-- Scenario S1
(*@\label{lst:ocl:var:derive:s}@*) context Task inv
    SameCharacteristicConstraint:
     inv: Stage.AllInstances()
         ->select(s|
             s.machines.forall(c | c.characteristics
                 ->includesAll(self.characteristics))
         ->includesAll(self.var(stage)))
-- Scenario S2
(*@\label{lst:ocl:var:derive:m}@*) context Task inv
    SameCharacteristicConstraint:
     inv: Machine.AllInstances()
         ->select(m| m.characteristics
             ->includesAll(self.characteristics)
         ->includesAll(self.stage.var(machines)))
-- Scenario S3
(*@\label{lst:ocl:var:derive:sm}@*) context Task inv
    SameCharacteristicConstraint:
     inv: Machine.AllInstances()
         ->select(m| m.characteristics
             ->includesAll(self.characteristics)
         ->includesAll(self.var(stage).var(machines)))
```

Listing 7: Annotated `SameCharacteristicConstraint` from **??** refactored around the annotations.

the tree **??**. Ideally, all the annotations should be at the top of the tree. The semantics of the expression gives clues to refactor them, the expression requires that: All the machines connected to a task (via a stage), each individually match the task's characteristics this is the same as requiring that: The set of machines that match the task, includes the set of machines connected to the task.

In 7 we can see the result of this rewrite for all three scenarios. The beginning of the expressions are now constant queries, and search for all the suitable machines (or stages), here isolated as `sel`:

```
let sel = Class->AllInstances().select(...) in
```

At the end of the expression we state that selection must include the result of the variable query over the machines and/or stage of the task:

```
sel->includesAll(...)
```

In Figure 5.3 we can see the AST resulting from the parsing of the expression of 7 Scenarios 2 and 3. The AST is significantly different to the previous one, but most importantly, the number of nodes within the scope of the solver is greatly reduced, to just navigation and the top level constraint. The CSP now only models the inclusion.

We applied this strategy manually with knowledge of the context, but it is generalisable. In the case of any constant sub-expression applied to a variable query, it is possible to determine candidates, or candidate sets, for that sub-expression and enforce the result of the variable query to be among them. For example, for: `self.var(ref).attrib<3` we can find candidates which satisfy the constant sub-expression `.attrib<3`. This adds more computation ahead of building the CSP, but also allows us to leverage the OCL engine in cases where it's more efficient such as this one.

5.4 Experimentation

→ **MC** Add this estimate: no refactoring, single interpreter, CharCSTR= 12k variables and constraints, applied to all tasks = 522k, +UMLCSP and the other constraints

5.4.1 Refactoring Expressions on our method

In our running example, our instance will have 4 stages (S) and 43 task (T), directly taken from [**?**]. We infer 24 machines (M) from their constraint model. In our annotation of `machines` we will choose 24 as the maximum cardinality of the reference, and thus have 24 pointer variables in the associated `AdjList`. Notice that the combinatorial complexity of the problem is quite challenging, since there are around 2.10^{40} possible graphs satisfying these assumptions. Generating all graphs and checking the OCL constraints would be unfeasible. The full code for this problem instance is available online.[4]

In Table 5.1 we find the metrics for the three RMS scenarios. Additionally to the same characteristic constraint, we also enforce the precedence constraint in the scenarios where the reference `Task.stage` is annotated. The first column identifies the annotations, and the scenarii. The second column gives the variable counts for each domain. The variable count includes all variables (problem and intermediate) involved in the expression. While the intermediate variable are unique to the model of this expression, the problem variables (tied to the instance objects) are shared by any other expression. Domains, informed by

[4] `https://github.com/ArtemisLemon/navCSP_RMSTaskConstraints`

var()	variables	domain	constraints	time
stage	43	S	(43)	0.1s
	0	M		
machines	0	S	(96)	0.1s
	96	M		
stage machines	43	S	2064 (+1032)	0.3s
	96+1032	M		

Table 5.1: Size and resolution times of the refactored Expressions

the third column, are identified by their upper bound, as the lower bound generally 0 for pointer variables. For the last row, we have both 96 problem variables of domain M, and because of the navigation from the 43 problem variables of domain S, each requiring their own copies of the variables of domain M, there are 1032 (43*N) intermediate variables.

In the constraints column, we count constraints of the OCL CSP, mainly element constraints and pointer arithmetic. In between brackets is the counter of *member* constraints. As *member* only propagates once, by default our solver doesn't include them in the constraint count. As it is the only constraint in two of these cases, we have included them.

Finally for times, we can see this problem is trivial for the solver. Most of the time taken is to build the model.

5.4.2 Refactoring Expressions in Alloy Models

```
1     abstract sig Characteristic {}
2     abstract sig Task { characteristics : set Characteristic ,..}
3     abstract sig Machine { characteristics : set Characteristic ,..}
4
5     abstract sig Stage {
6         ..,
7         tasks: seq Task ,
8         machines: seq Machine
9     }
```

Here we have a stripped down Alloy specification of the RMS metamodel. → **MC** to be as close to our model, sequence variables `seq` are used for the problem variables, Task<->Stage<->Machines, and sets (which are much more efficient) are used for part of the constraint expression we interpret before building the CSP

```
fact MachineContainment {
    all s1,s2 : Stage | s1!=s2 => s1.machines.elems & s2.machines.
        elems=none
}

fact OppositeTaskStage {
    all t : Task | t in t.stage.tasks.elems
    all s : Stage | s = s.tasks.stage.elems
}
```

These two facts model the notions of UML Containment and Opposite relations. The containment constraint states *for any two stages, if they are different then the intersection of their respective sets of machines is empty.* The opposite constraint comes in two parts. The first one states *for any task, the stage its associated to, is associated to it.* The second one states *for any stage, the tasks its associated to, are each associated to it.*

```
one sig Stage10 extends Stage{} {ID=10 maxMachines=3}
one sig Stage20 extends Stage{} {ID=20 maxMachines=3}
one sig Stage30 extends Stage{} {ID=30 maxMachines=3}
one sig Stage40 extends Stage{} {ID=40 maxMachines=3}
```

Along with the alloy specification of the metamodel, we can specify a partial model. In this listing we input the existing model data, by defining a class (signature) for each object instance. The object will be the only instance of its distinct subclass (defined by the known data), which inherits from its class.

```
// 2994706 vars. 17315 primary vars. 10035908 clauses. 5409ms.
// Instance found. Predicate is consistent. 7393ms.
fact SameCharacteristicConstraint{
    all t: Task | t.characteristics in t.stage.machines.
        characteristics.elems
}
```

```
// 2749262 vars. 17315 primary vars. 9551642 clauses. 4812ms.
// Instance found. Predicate is consistent. 2427ms.
fact SameCharacteristicConstraint_Refactored{
    all t: Task | t.stage.machines.elems in { m:Machine | t.
        characteristics in m.characteristics }
}
```

In the two listings above, we have both forms of the *same characteristic constraint*, and the solver output as comment. The variable and clause count of the output includes the rest of the model, such as the *precedence constraint*, and UML specific constraints such as the *UML Opposite Relation*, *UML Containment Relation*. Without this constraint the model has 2611962 vars, 17315 primary vars, and 9305982 clauses. Meaning the first version of the expression requires 382744 vars and 9105316 clauses, The refactored version of the expression requires 137300 vars and 8621050 clauses, which is 245444 vars and 484266 clauses less than the original, 5.3% the original number of variables and 94.6% of the original number of clauses.

Most notable is the time difference, the first version of the expression takes around 8 seconds to solve, while refactored version around 3 seconds. The times in the comments are the best times among a few reruns.

5.5 Discussions

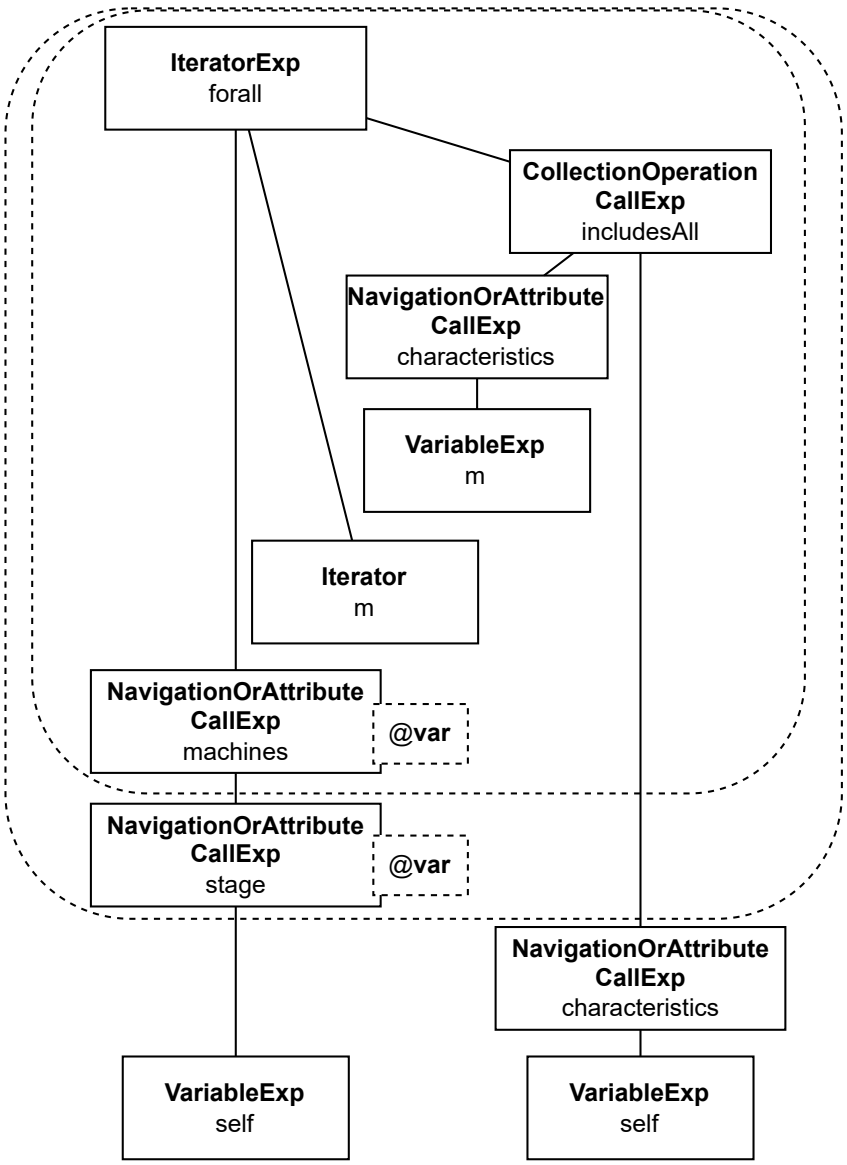


Figure 5.2: AST of `SameCharacteristicConstraint` from **??** Scenario S1, S2 & S3. [AST]

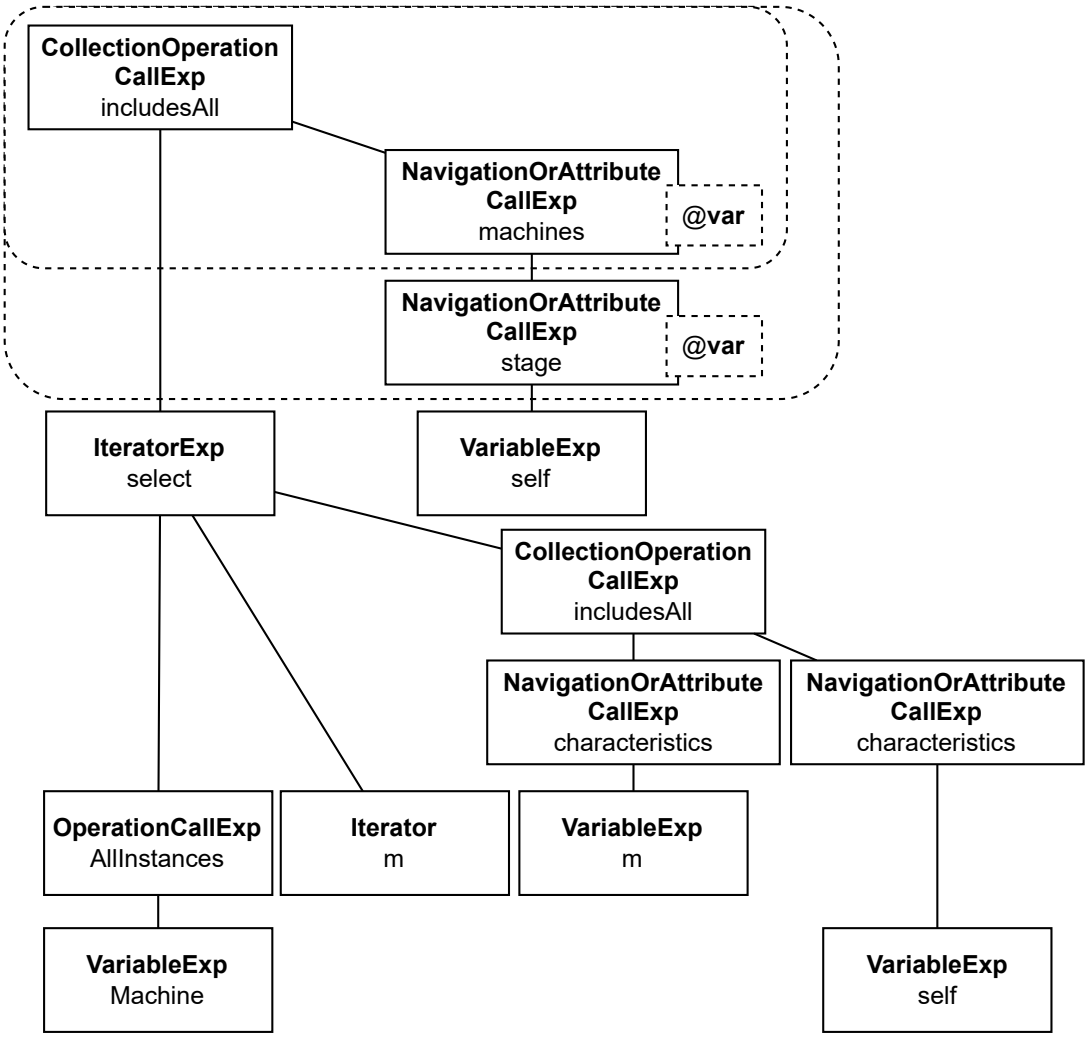


Figure 5.3: AST of SameCharacterticConstraint from 7 Scenario S2 & S3 [AST]

Chapter 6

Contribution : UML CSP

In this chapter we present the encoding of the UML model as a constraint satisfaction problem.

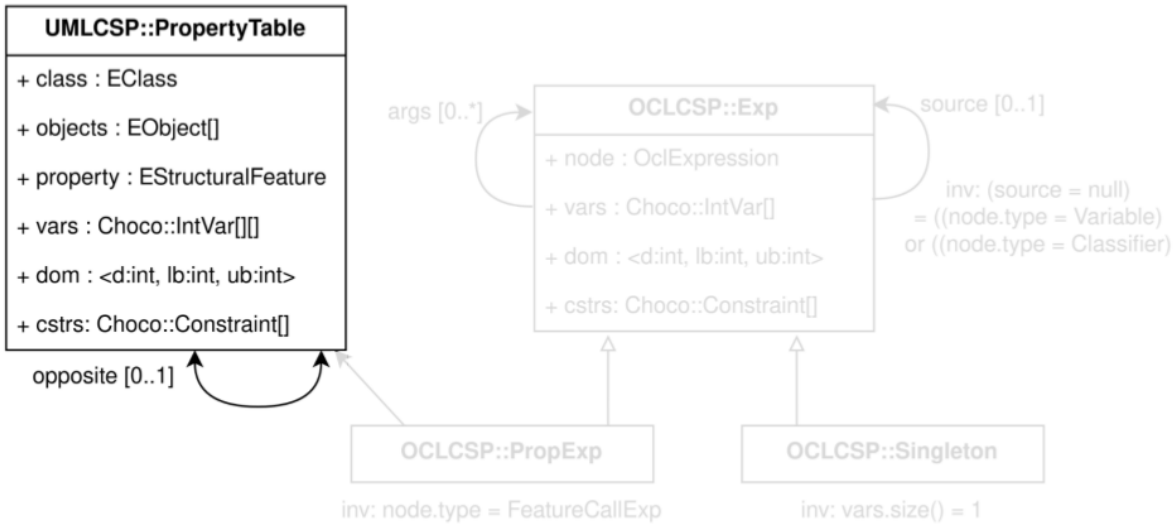


Figure 6.1: Property Tables in our metamodel of UMLCSP

We can model Integer Collections type properties using arrays of integer variables. It is possible to encode a lot of finite domain problems for different property types. This encoding works for both integer attributes and object references.

References has have *extra layers* of models, leveraging the limited domains for pointer variables

Object.attribute			
$Object_i$	**attribute**		
0	d	d	d
1	-3	4	6
2	1000	a_{22}	a_{23}
3	-99	-33	a_{33}

Object.reference		
$Object_i$	**reference**	
0	nullptr	nullptr
1	3	2
2	ptr_{21}	ptr_{22}
3	1	ptr_{32}

Table 6.1: Encoding of the instance from Figure 1.3 as tables of integer variables

6.1 Encoding Properties

A. Encoding Properties. The variables represent the properties–attributes and references–of the objects in the instance. Each class property is encoded as a matrix of integer variables, denoted $Class.property$. Each row in this matrix corresponds to one object of the class; for example, the i-th row is noted as $Class_i.property$ where $i \in [1, o]$ and $o = |Class|$ is the number of objects of that class. The number of columns p in this table is derived from the property's cardinality, which is given by n and m from Figure 7.1.

$$Class.property = \{x_{11}, ..., x_{op}\}$$

$$\forall x \in Class.property,\ domain(x) = \{d\} \cup \{lb..ub\}$$

Each property variable x_{ij} in the matrix has a domain defined by a lower bound lb, an upper bound ub, and a special dummy value d, where we set $d = lb - 1$. The property type, e.g., reference or attribute, determines the specific domain bounds. Attributes with an integer type may require large ranges, making domain enumeration impractical. This limits our ability to use certain global constraints like $global_cardinality(c)$onstraint, which counts the occurrences of domain values and therefore require finite, reasonably small domains. By default we chose a 16-bit range for these values: $lb = -32768$ and $ub = 32767$, meaning $d = -32769$, but these bounds can be refined by annotating the model accordingly [**?**]. For reference properties, the domain is defined as $\{1, \ldots, o\} \cup \{nullptr\}$, where o is the number of instances of the target class. These variables, named `ptr`, acts as pointers: values in $[1, o]$ identify object rows, and 0 (i.e., dummy value <u>nullptr</u>) denotes the absence of a reference. To support <u>nullptr</u>, an extra row is added to each table to represent a dummy object.

Table 6.1 shows the encoding of the instance from Figure 1.3, assuming $n = 2$ and $m = 3$ from the metamodel in Figure 7.1. The left side represents attributes, while the

right side represents references. Each object (plus one dummy object) gets a row. The attribute variables a_{ij} are assigned the domain $\{lb, \ldots, ub\} \cup \{d\}$, and reference variables ptr_{ij} are assigned the domain $\{1, \ldots, o\} \cup \{nullptr\}$ with $o = 3$.

Model construction proceeds in two steps. First, we create matrices of variables with their full domains. Second, we instantiate some of these variables using data from the actual instance. In our current setting, we assign the exact values from the instance. Variables that remain uninstantiated may either be assigned dummy values or left free to explore during search, depending on the objective of the analysis. To choose between these behaviors and to reduce the size of the CSP, in previous work we've proposed an annotation system for OCL [**?**], which allows the user to identify variables. These annotations split the OCL expressions into parts which can be dispatched between our CP interpretation, and that of a standard interpreter. This reduces the scope and size of the CSP, notably in terms of modeled properties. If the domain of the problem variables is limited and order doesn't matter, this encoding can suffice to model properties.

6.1.1 Additional *enumerated* encoding for References

References are a special case of property, with a grately restricted domain. The values in the domain of pointer variables identify rows of property tables. The domain of relation variables is generally small enough to be enumerated. Therefore, we give them accompanying variables counting the occurrences for each value of the domain.

$$Class.propOcc = \{occ_{00}, ..., occ_{oq}\}$$

$$\forall x \in Class.propOcc,\ domain(x) = \{0..q\}$$

We similarly declare a table of variables, where o is the number of objects of the class, but q is the number of objects in the target class. The domain of these variables is These variables are constrained to the property variables with the *global cardinality constraint gcc*:

$$gcc(Class_o.property, Class_o.propOcc)$$

The *gcc* constraint in applied to each object (or row) individually. We can also use this encoding and constraint to apply set constraints, and to navigate the graph with multiplicity.

6.2 Constraint Models for UML Reference Types

UML offers an array of specific reference types, an important one for our models is `Opposite`. An example of opposite relation is the parental between two `Person`: the opposite of `Person.child` is `Person.parent`. Using the *occurrence variable encoding* of the two references' property tables, we simply apply a *if it occurs in one, it occurs in the other*:

$$A_i.refOcc_j, \neq 0 \iff B_j.oppOcc_i \neq 0$$

s

Additionally UML offers a containment relation which can easily be modeled with an additional *gcc* on the whole property table, for which the occurrence variables domains are 0/1. To do containment on a self-referential reference, we need to add the CSP equivalent to `self.ref.closure().excludes(self)`.

6.3 Constraint Models for UML Collection Types

As described in Section **??**, properties in class diagrams (e.g., Figure 7.1) can be annotated with collection types: `Sequence`, `Bag`, `Set`, or `OrderedSet`. These types can be enforced through constraint models to ensure consistency and reduce symmetries in the data..

For `Sequence`, permutations of the same multiset (e.g., $\{1, 1, 2\}$) yield distinct sequences. However, in our encoding, sequences such as $\{1, 2, d, 1\}$ and $\{1, 2, 1, d\}$ are treated as equivalent, since they encode the same effective ordering of values (e.g., the position of the dummy value d is ignored). To correctly model sequences, we impose an ordering where all dummy values are grouped at the end.

Let $X = \{x_1, .., x_p\}$ be the variable array for a property in the matrix $= Class.property$. The `Sequence` constraint is defined as: $Sequence(X) \iff \forall i \in [1, p[, (x_i = d) \Rightarrow (x_{i+1} = d)$. This ensures dummy values appear only at the end. We reformulate it using the `regular` global constraint applied to a Boolean mask $S = \{s_1, \ldots, s_p\}$:

$$Sequence(X) : \begin{cases} regular(S, DFA) \\ s_i = [\![x_i \neq d]\!] \end{cases} \tag{6.1}$$

The automaton *DFA* (Figure 6.2) accepts patterns of the form 1^*0^*, ensuring non-dummy values precede dummy ones. Here, S acts as a mask distinguishing actual values

(1) from dummies (0) while avoiding symmetry.

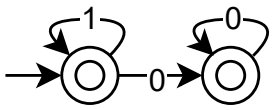


Figure 6.2: DFA packing dummy values for instance variables

For the `Bag` and `Set` types, all permutations of values are considered equivalent. To remove ordering symmetries, we sort the values in decreasing order, effectively pushing dummy values to the end:

$$Bag(X) : \Big\{ \forall i \in [1, p[: x_i \geq x_{i+1} \tag{6.2}$$

To model `Set`, we additionally enforce uniqueness among non-dummy values. Indeed, the sequence $\{d, d, d\}$ would be interpreted as an empty set. To this end, we define a relaxed variant of the `alldifferent` global constraint:

$$\begin{aligned} & \mathit{alldifferent_except_d}(X) \\ \iff & \forall\, i, j\, (i < j) \in [1, |X|], (x_i \neq x_j) \vee (x_i = x_j = d) \end{aligned}$$

$$Set(X) : \begin{cases} \mathit{alldifferent_except_d}(X) \\ Bag(X) \end{cases} \tag{6.3}$$

For `OrderedSet`, both value order and uniqueness matter. We combine the constraints used for `Sequence` and `Set`: dummy values must be packed at the end, and non-dummy values must be pairwise distinct. Formally:

$$OrderedSet(X) : \begin{cases} \mathit{alldifferent_except_d}(X) \\ Sequence(X) \end{cases} \tag{6.4}$$

This ensures a well-formed sequence without repetitions, where dummy values are ignored in uniqueness checks and appear only at the end of the array. These CP encodings ensure that model properties respect their specified UML and OCL collection types, enabling correct interpretation and reducing symmetry in instance generation.

6.4 Using information from Variable Annotations

Chapter 7

Contribution : OCL Navigation

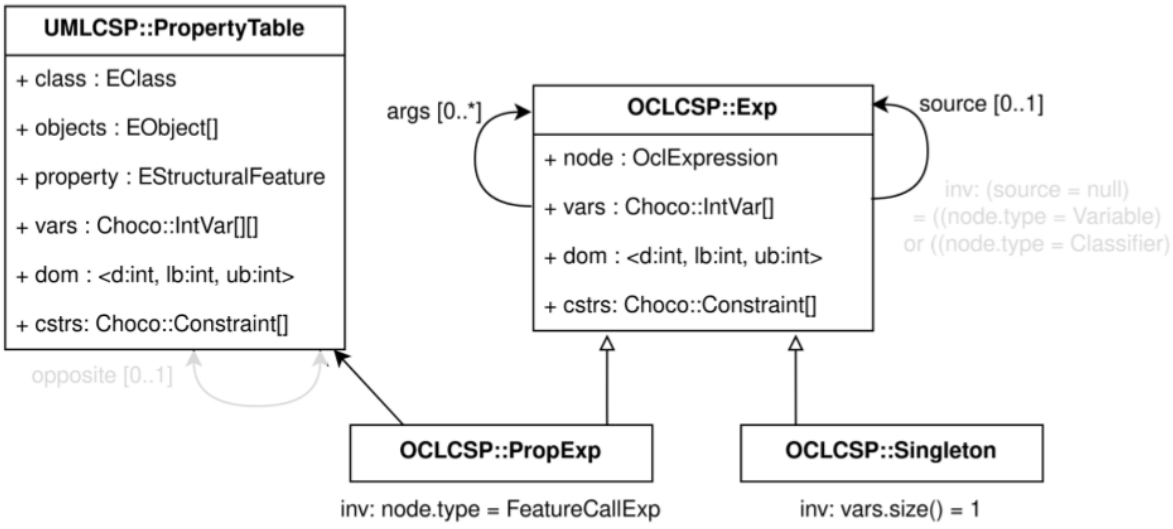


Figure 7.1: navCSP in our method

In this chapter we'll be modeling the link between the UML CSP modeling the instance, and the OCL CSP modeling the constraint expressions. We'll be considering expressions such as `src.prop` and `src.var(prop)`, which means also connecting with the interpretation of the *constant expressions* which can be done independently. When building the model for navigation we can assume to have access to all the information accessible to a `PropExp`, such as the domains for the variables of the `src` expression and the `PropertyTable` identified by `prop`.

We will first show and test the main model for this thesis, which preserves order. Then we'll present a secondary model which only preserves multiplicity.

7.1 CP model for OCL queries on the instance

Querying an instance involves navigating the object graph through references and retrieving attribute values. In OCL, navigation refers to the operation that, given source collection of objects and a reference property, returns a collection of target objects through that reference. We conflate this with attribute operations–as defined in the OCL specification–which return a collection of attribute values from a source collection of objects. In our encoding, both navigation and attribute access results are uniformly represented as integer variables.

Consider an OCL expression of the form `src.property`, where `src` is itself an expression like `self.reference` or `self.reference.reference`.

Let $Ptr = \{ptr_1, ..., ptr_z\}$ be the variables encoding the evaluation of `src`, with $dom(ptr_i) = \{1..o\} \cup \{nullptr\}$. Let T be the flattened array representing the `Class.property` matrix for `property`, where `property` refers to either an attribute or reference of the referenced class. Let p be the number of columns in the matrix. Let $Y = \{y_1, \ldots, y_{z \cdot p}\}$ be the variables representing the result of `src.property`. To link Y with T and Ptr, we define the navigation constraint:

$$nav(Ptr, T, Y) \iff \forall i \in [1, z], \forall j \in [1, p] : y_{(i-1)p+j} = T_{ptr_i \times p+j}$$

This constraint links the source pointers to the appropriate rows in the property table. This is reformulated in CP as a conjunction of *element* constraints, using intermediate variables for encoding the pointer arithmetic ($ptr_i \times p + j$):

$$nav(Ptr, T, Y) : \begin{cases} \forall i \in [1, z], \forall j \in [1, p] : \\ \quad ptr'_{ij} = ptr_i \times p + j \\ \quad element(y_k, T, ptr'_{ij}),\ k = (i-1)p + j \end{cases} \tag{7.1}$$

The intermediate variables introduced are functionally dependent on the Ptr variables and do not require enumeration during search. Given Ptr and T, the value of Y can be determined. However, given an instantiation of Y, this model cannot fully determine Ptr and T, but it can filter to some extent. Thus, OCL query variables depend on the instance variables, and a query result may correspond to multiple instances.

It is important to note that the intermediate variables introduced by this reformulation are functionally dependent on the Ptr variables of the constraint. This means, we do not

Object.reference.attribute						
$Object_i$	**reference.attribute**					
1	-99	-33	a'_{13}	1000	a'_{15}	a'_{16}
2	a'_{21}	a'_{22}	a'_{23}	a'_{24}	a'_{25}	a'_{26}
3	-3	4	5	a'_{34}	a'_{35}	a'_{36}

Table 7.1: Encodings of `self.reference.attribute` for all objects of Figure 1.3 as a table of integer variables

need to enumerate upon these variables during search. Similarly, given an instantiation of Ptr and T, in the context of model verification for example, we can determine Y. However, given an instantiation of Y, this model alone cannot determine Ptr and T, but it can filter to some extent. In the overall CSP this means that the variables encoding the OCL queries are all functionally dependent on the instance variables, but a query that solves the problem isn't associated with one instance.

Table 7.1 shows the results of the query `self.reference.attribute` using the navigation CP model (7.3) on the instance from Table 6.1. Result variables a'_{ij} share the same domains as a_{ij} but follow the reference and attribute order, introducing gaps due to ordering, e.g., a'_{13} might be the third variable, but yield a different third value (e.g., 1000) if $a'_{13} = d$. Similar effects occur in other OCL reformulations like union and append. Despite these gaps, value order and duplicates are preserved. These outputs are interpreted using the same models used for casting to collection types, such as `asSequence()`, discussed in Section **??**.

Table 7.1 shows us the result of query `self.reference.attribute` using the navigation model CSP 7.3 on the instance from Table 6.1 The variables in this table, noted a'_{ij}, have the same domain as the a_{ij} variables. We also find the instantiated values, with respect to the order in the reference and the attribute: the variables of the first referred object come first, in the same order as in their original table. This introduces gaps between values, as illustrated by the first line: the third variable is a'_{13}, but if $a'_{13} = d$ the third value is 1000. Our reformulations of other OCL operations, such as union and the sequence operation append, similarly introduce gaps. However, despite these gaps, the order and multiplicity across values are preserved. The models to get a correct interpretation these collections are the same as the models to cast to a collection type, such as `asSequence()`, and are explored in Section **??**. We will therefore need models to interpret these as the correct collection type.

7.2 NavCSP experimentation

Navigating a model adds a great deal of complexity. The pointer navigation Equation 7.3 is the greatest factor in that complexity. It takes effect in variable query expressions such as: `src.var(ref).prop` where we want to find a property based on variables in the scope of the solver. Whether the property is variable or not, or is an attribute or a reference, the same navCSP applies. In the case the property is a reference, we can chain the CSP, which greatly increases complexity.

OCL Query Dimensions

To evaluate the navigation provided by Equation 7.3, we will look at the size of the CSP modeling the following OCL expression:

```
let query = self.ref.ref...ref in
```

Such that `self.ref` is reflexive variable reference, modeled with N pointer variables, identifying objects of the same type. The depth of the navigation, is noted d, with $d = 0$ as the case of variable property access, `query = self.ref`. Adding further navigations increments d, for example $d = 2$ corresponds to `self.ref.ref.ref`.

OCL Query Size

In Figure 7.2 we can see the number of query atoms, meaning equally: the intermediate pointer variables, element constraints or pointer arithmetic required to model this query, which is found using the formula:

$$f(N, 0) = 0$$

$$f(N, d) = f(N, d - 1) + N^{1+d}$$

1) No matter the size of the `AdjList`, the first annotated reference implies no intermediate pointers, as we simply find the problem variables associated to `self`.
2) If we are to navigate deeper, we make an additional hyper-table of intermediate variables, indexed by the prior lower dimension table of pointers. To examine the formula, let's look at the case of $d = 1$, or `self.ref.ref`:

$$f(N, 1) = 0 + N^2$$

We have N pointers coming in from `self.ref`, and they each point to N pointers. Resulting in a table of intermediate pointer variables. If we navigate deeper, let $d = 2$:

$$f(N, 2) = 0 + N^2 + N^3$$

For every pointer in the previous table N^2, we associate N more pointers. Giving us now a hyper-table, cubed. If we navigate deeper, the 3D hyper-table will similarly index a 4D hyper-table.

The graph in Figure 7.2 starts at 1 on the x,y axes or $f(1, 1)$, which gives 1 on the z axis (log scale). For a single navigation from a single pointer variable (AdjList of size 1), we have a single query atom. For a single navigation from an `AdjList` of size 10 or $f(10, 1)$, we have 100 query atoms. For `AdjList` variables of size 1, navigating with a query depth of 10 or $f(1, 10)$, results in 10 query atoms.

On the left background, we can see the curve resulting from increasing `AdjList` size. While on the right, we can see the curve resulting from increasing navigation depth. We can see from this that increasing the navigation seems to increase the size of the problem logarithmically, while increasing the number of pointers for a reference is exponential.

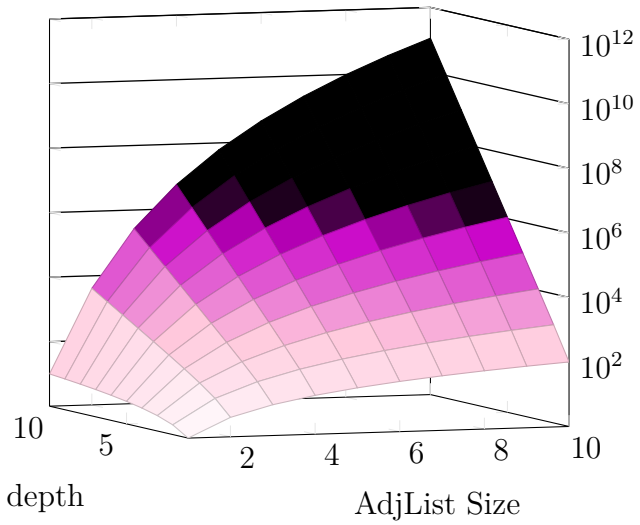


Figure 7.2: Number of query atoms in relation to `AdjList` size and navigation depth [AST]

The complete navigation model has twice as many constraints $2f(N, d)$, as we need both an element and some pointer arithmetic for each intermediate variable. Our implementation of the pointer arithmetic implies an additional intermediate variable, giving a

total of $2f(N, d)$ intermediate variables.

The total number of propagations required to find all counter proofs, or validate a model also aligns with the number of constraints found here $2f(N, d)$, validation would correspond to all the problem variables having only one possible value. While it is a large number it's still fast to run all these propagators once, and running out of memory space for the model became a more limiting factor than time in our tests.

Going beyond validation, and searching for a model fix, or completing a model such as in our use-case, means increasing the domains of the problem variables and by consequence the intermediate variables, and in the case of model completion having the full range of possibilities for all these variables.

Subset Sum Problem

[1] by applying the following constraints to the query from a single object (among up to 120), we can model a variation on the subset sum problem:

```
query->sum(attribute) = Target

and query->isUnique(attribute)
```

Where `self.attribute` of an object is a constant integer attribute between 10 and 29. All of them together forming a multiset, from which we'll find a subset with the right sum. Initial testing with this problem gives fast non-trivial solutions, up to a few minutes, for queries with up to around 10^4 intermediate variables. When no subset sums equal the target, such as finding a subset summing to 1, or when solving for trivial targets such as 0, the process takes less than a few minutes up to 10^6 intermediate variables. Bigger problems reached our memory limit. These results color Figure 7.2, the lightest area being quickly solvable, the darker area being quickly verifiable and the black area being too big to model.

[1] `https://github.com/ArtemisLemon/navCSP_SubsetSum`

7.3 Additional navigation model using occurrence variables (bag navigation)

In this model M is the adjacency matrix we're using to navigate. There are r rows and c columns in M. This model can be seen as adding a *virtual row* Occ' to M. There are c variables in Occ'. The source expression variables Occ, can be a row or a *virtual row* of a previous matrix M', or M in the case of a reflexive reference. There are r variables in Occ. If the relation is reflexive, $r = c$.

Each variable in the new *virtual row*, sums the column it corresponds to, for which each variable is multiplied by the corresponding variable from Occ.

$$nav(Occ, M, Occ') : \begin{cases} \forall j \in [1, c] : \\ Occ'_j = sum([Occ_i * M_i j | i \in [1, r]]) \end{cases} \tag{7.2}$$

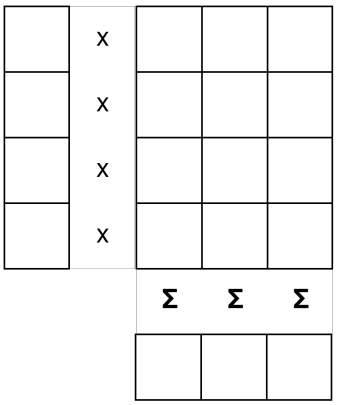


Figure 7.3: Diagram of *bag* navigation

In this diagram illustrates how the variables and constraints are organized spatially. The incoming array Occ is vertical to the left, the output array Occ' is horizontal at the bottom. We can see how each variable of Occ multiplies a row of M, and each variable in Occ' sums the column.

This additional model can be tied to the base model using *gcc* in the same way as described in the UML CSP section. However it can entirely replace the base model, when order doesn't matter, which can be detected by the interpretation process.

7.3.1 Closure

We will also use the adjacency matrix to model closure on a reflexive relation. We can get trivially get a 0/1 matrix from the adjacency matrix preserving multiplicity. Because the relation is reflexive, the adjacency matrix is square, of side size z

$$closure(X, M, Y) : \begin{cases} \forall j \in [1, z] : \\ Y_j = \bigvee([(X_i \wedge M_i j) \vee (Y_i \wedge M_i j) | i \in [1, z]]) \end{cases} \tag{7.3}$$

This model uses both the input and output to activate the column for the output variables.

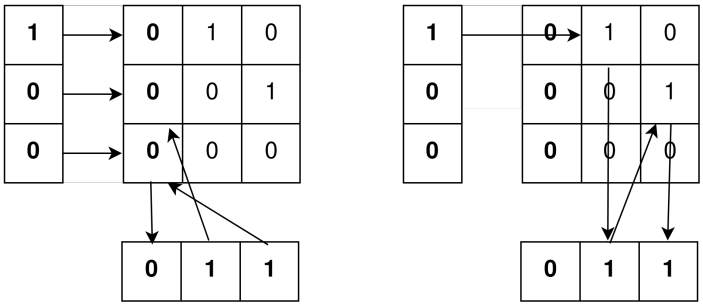


Figure 7.4: Diagram of closure solution

In this figure the arrows towards the matrix are the *and* between the values of the column and the input or output, and the arrow towards the output is the *or* on the column. In this diagram we see an expression equivalent to `self.ref.closure()` where `self` points to the first object, and `ref` is a reflexive reference. The source occurrences point to the first object, but no relation in the `refTable` goes towards the first object. Hence, the first object isn't in the output. However the first object points to the second, and having the second in the output gets us the third in the output.

Chapter 8

Contribution : OCL CSP

In this chapter we'll cover the bulk of the CSPs which model OCL operations.

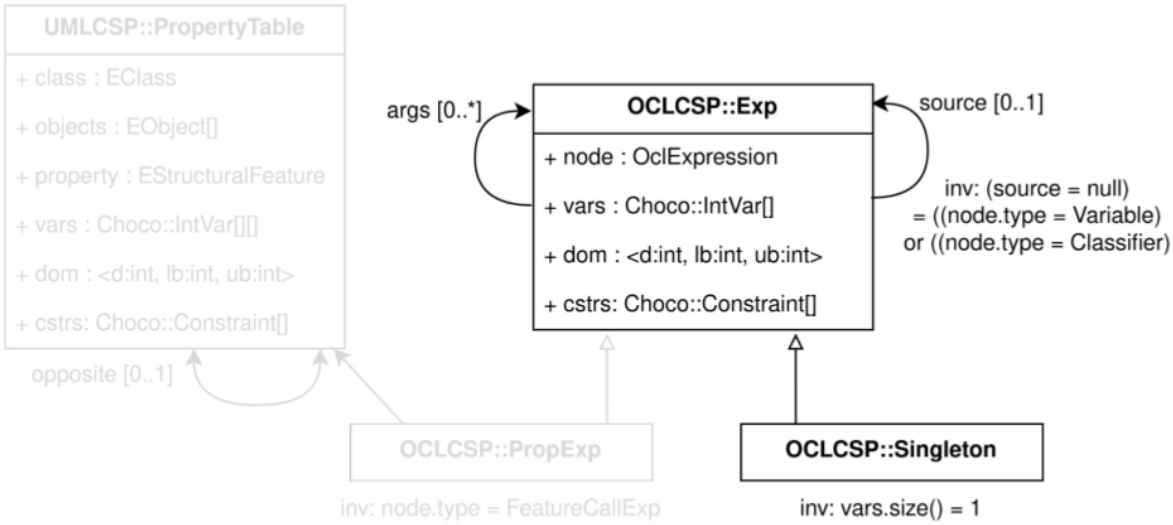
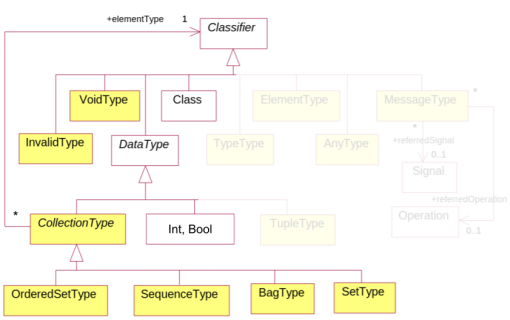
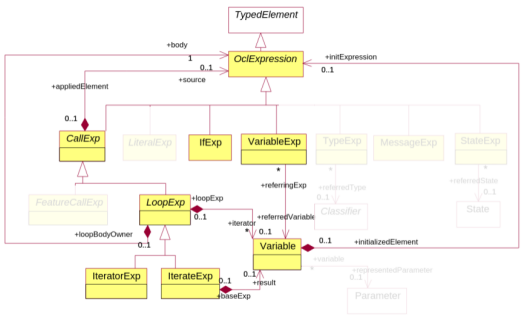


Figure 8.1: metamodel of expressions in OCLCSP



((a)) OCL Type Coverage



((b)) OCL Expression MMCoverage

src \out	bool	int	element	collection	Sequence	Bag
bool	and					
int	<	+				
collection	=, <> includes, forall	size count	any	select reject	asSequence sortedBy	asBag
Sequence			first at		subSequence union	
Bag						union intersection

The first section covers integer and boolean operations. Collection operations are split across result types: operations of section 2 result in integer, section 3 in booleans, section 4 in collections. Collection type casting operations are in section 5, and typed collection operations are found in the following sections.

8.1 CP Models for OCL Integer and Boolean Operations

While in many cases modeling OCL arithmetic and logic on attributes is straight forward, there are edge cases we must consider. The reason for this is UML allowing for optional properties: `[0..1] Boolean` and `[0..1] Int`. For situations where the property is required, we can simply use arithmetic and logic operations.

8.1.1 3-valued logic for OCL

OCL is fundamentally a 4-valued logic, with values: true, false, void, and invalid. Both Void and invalid behave similarly for the logical operations, and we have chosen to combine them in our encoding. However invalid expressions can often be statically detected, and stop the interpretation before building the CSP. OCL# also collapses OCL to a 3-valued logic using the empty set $\emptyset$ to encode void and invalid expressions. → **MC** because RFOL

a	b	a and b	a or b	a xor b	a implies b	not a
2	2	2	2	0	2	0
2	1	1	2	1	1	0
2	0	0	2	2	0	0
1	2	1	2	1	2	1
1	1	1	1	1	1	1
1	0	0	1	1	1	1
0	2	0	2	2	2	2
0	1	0	1	1	1	2
0	0	0	0	0	2	2

In this table we find the semantics for boolean operations in OCL, combining both invalid and void into *maybe*. For this table we have chosen $\top$=2, *maybe*=1 $\bot$=0. The semantics for these logical operations encoded with integers can be simply modeled with integer operations:

- $\neg a \equiv 2 - a$
- $a \wedge b \equiv min(a, b)$
- $a \vee b \equiv max(a, b)$
- $a \rightarrow b \equiv max(2 - a, b)$

- $a \oplus b \equiv min(2 - min(a, b), max(a, b))$

We can see these models employ *min* and *max* as *and* and *or*, and $\top - x$ for not, and model *implication* and *the exclusive or* on top:

- $a \rightarrow b \equiv (\neg a \vee b)$
- $a \oplus b \equiv (\neg(a \wedge b) \wedge (a \vee b))$

8.1.2 Property encoding to 3-valued logic for optional Booleans

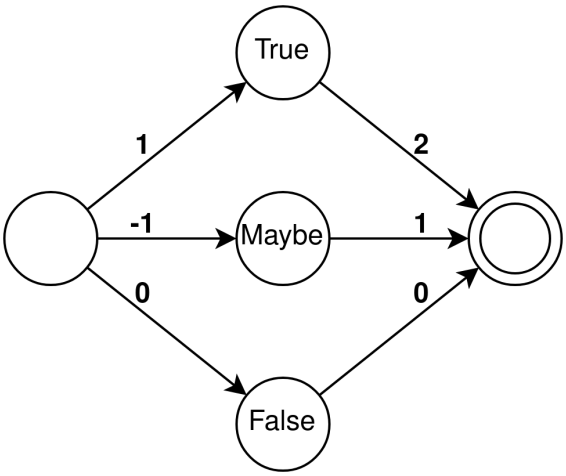


Figure 8.3: Part of the tree for `src.forall(a,b,c| (a<b and b<c) implies (a+b=c+c))`

In our base encoding for properties, True equals 1, False equals 0, and no information is -1. We could implement a different encoding for booleans, or simply convert from one encoding to the other using an regular expression constraint based on the AFD here.

8.1.3 Operations with Optional Integers

In the OCL Specification: *The interpretation of operations is considered strict [...]. Hence, an invalid or null argument value causes an invalid operation result. This ensures the propagation of error conditions.* This means expressions such as `2 + null` are *invalid*, and generally invalidate the whole model.

To follow the specification and and only handle valid OCL, we can fail by making adding a contradiction to the constraint model, essentially adding AND False appropriately.

Taking addition as an example: $x + y = z$, becomes $x + y = z \wedge x \neq d \wedge y \neq d$, to fail because invalid expression: 2 + d = False. In this case: models with data and solver configurations forcing an absence will be invalid and the will be no solution to the CSP, otherwise it will populate all the attributes with values.

However, we can also relax the arithmetic. If we didn't want to force the presence of attributes or fail, and wanted to maybe ignore operations on absent values, we can exchange the d value of a variable with another. For instance $x + y = z$, becomes

$$x'+y' = z \wedge (x = d => x' = 0) \wedge (x \neq d => x' = x) \wedge (y = d => y' = 0) \wedge (y \neq d => y' = y)$$

Here, if either x or y are null, we replace their value with 0 in x' and y', skipping the addition. We could always interpret null as 0, or choose an interpretation which skips the operations, such as 1 in multiplication, *infinity* in *less than*, etc..

→ **MC** OCL# : in OCL- there is no invalid or null, and arithmetic isn't done with missing values? Check this.

8.2 CP Models for OCL Collection to Int Operations

In this section we'll consider expressions of the type `src.op()` where `src` is a collection and the result of the operation is a single integer.

8.2.1 size()

Consider the expression `src.size()`, where `src` is an expression resulting in a collection: the `size` operation returns the size of the `src` collection.

Because of our encoding with collections using dummy variables, we define the size constraint around them, where X are the variables representing `src` and y is the output of the `size` operation:

$$size(X, y) \iff y = \sum_{0<i\leq|X|} [\![x_i \neq d]\!] \tag{8.1}$$

This definition states that y is equal to the count of values not equal to d. Reformulation using Global Constraints:

$$size(X, y) : \begin{cases} count(d, X, y') \\ y = |X| - y' \end{cases} \tag{8.2}$$

In this model, we simply count the number of times the dummy value d appears in the source collection X. The output y is constrained to equal the difference between the size of the variable array X, and the count of d: $y = |X| - y'$.

For example, given the collection $[3, 1, 2, d, d]$ we count 2 d, the size of the array is 5, so the number of values in the collection is 5-2=3.

8.2.2 sum()

Consider the expression `src.sum()`, where `src` is an expression resulting in a collection of integers: the `sum` operation returns the sum of the `src` collection.

Because of our encoding with collections using dummy variables, we define the sum constraint around them, where X are the variables representing `src` and y is the output of the `sum` operation. The sum operation can be defined similarly to `size`, by additionally

multiplying by the value of each variable of the source:

$$sum(X, y) \iff y = \sum_{0<i\leq|X|} x_i * [\![x_i \neq d]\!] \tag{8.3}$$

This definition states that y is equal to the sum of values not equal to d. Among the globals there is a sum constraint which we can employ. Reformulation using Global Constraints:

$$sum(X, y) : \begin{cases} count(d, X, s') \\ y = sum(X) - ds' \end{cases} \tag{8.4}$$

In this model use $y = sum(X)$, and then simply subtracts the dummies.

For example, given the collection $[3, 1, 2, d, d]$ we count 2 d. The sum of the array being $6 + 2d$, we use the count to negate the occurrences of d: $6 + 2d - 2d$.

8.2.3 count(e)

Consider the expression `src.count(e)`, where `src` is an expression resulting in a collection and `e` is an expression resulting in a single element: the `count` operation returns the count of `e` in `src`. With X as the variables representing `src`, y as the variable representing `e`, and z representing the count, we can define the operation as follows:

$$count(X, y, z) \iff z = \sum_{0<i\leq|X|} [\![x_i = y]\!] \tag{8.5}$$

Meaning simply applying the appropriate global:

$$count(X, y, z) : \left\{ count(y, X, z) \right. \tag{8.6}$$

8.2.4 max()

Consider the expression `src.max()`, where `src` is an expression resulting in a collection of integers: the `max` operation returns the highest value in the `src` collection:

$$max(X, y) \iff (\forall x \in X, y \geq x) \wedge (\exists x \in X, x = y) \tag{8.7}$$

Reformulation using Global Constraints:

$$max(X, y) : \Big\{ maximum(y, X) \tag{8.8}$$

Because for our encoding using the minimum of the domain as the dummy it doesn't effect the constraint model.

8.2.5 min()

Consider the expression `src.min()`, where `src` is an expression resulting in a collection of integers: the `min` operation returns the lowest value in the `src` collection.

Because of our encoding with collections using dummy variables, and using the lowest value of the domain as dummy value, our definition of minimum needs to exclude that value:

$$min(X, y) \iff (\forall x \in X, x \neq d \iff y \leq x) \ \wedge (\exists x \in X, x = y) \ \wedge y \neq d \tag{8.9}$$

In order to employ the *minimum* global constraint, we create a copy of the list, where variables equal to d get flipped to the beyond the upper bound of the domain, giving us the following reformulation using Global Constraints:

$$min(X, y) : \begin{cases} \forall i \in [1..z] \\ \quad x'_i = x_i - (2d + 1) * [\![x_i = d]\!] \\ minimum(y, X') \end{cases} \tag{8.10}$$

For example, given the collection $[3, 1, 2, d, d]$ we create the copy flipping the dummy: $[3, 1, 2, -2d + 1, -2d + 1]$, because d is the smallest value of the domain and $-2d + 1$ is just above the domain of valid values in our encoding.

8.3 CP Models for OCL Collection to Bool Operations

In this section we'll consider expressions which result in a single boolean. It is common to know the resulting boolean at compile time, such as at the top of an invariant expression. In the cases where the resulting truth value is known, we often have a simpler model.

8.3.1 equals and different

Consider an expression of the form `exp = exp2`, where both expressions resolve to a collection. If one array is smaller than the other, we append variables instantiated to d. To model equals and different in OCL, we can simply use a conjunction or disjunction.

$$coleq(X, Y) : \begin{cases} \forall i \in [1..z] \\ \quad x_i = y_i \end{cases} \tag{8.11}$$

$$coldiff(X, Y) : \begin{cases} \exists i \in [1..z] \\ \quad x_i \neq y_i \end{cases} \tag{8.12}$$

In expressions such as `(exp = exp2) implies exp3`, the equality between collections is no longer enforced, but checked. In other words, we need a reified version of this constraint model.

$$coleq(X, Y, z) : \begin{cases} \forall i \in I \\ \quad b_i \iff (x_i = y_i) \\ minimum(B, z) \end{cases} \tag{8.13}$$

In this first model for equals, we reify the individual equalities. The reified value for this constraint model, is the minimum value for the reified equalities.

$$coldiff(X, Y, z) : \begin{cases} \forall i \in I \\ \quad b_i \iff (x_i \neq y_i) \\ maximum(B, z) \end{cases} \tag{8.14}$$

In this model for not equal, we reify the individual inequalities. The reified value for this constraint model, is the maximum value for the reified equalities.

For example, given the collections $X = [3, 1, 2, d, d]$ and $Y = [3, 0, 2]$, we pad the

second with d, to make it the same length as the longer array: $Y' = [3, 0, 2, d, d]$. Because $x_1 = 1$ and $y'_1 = 0$. These two collections are different. If $y_1 = 1$, they would be equal.

When reifying, for each couple (x_i, y'_i) we reify the equality to b_i: $B = [1, 0, 1, 1, 1]$. The minimum of this array z gives us the truth value of this reified constraint.

8.3.2 includes(e)

Consider the expression `src.includes(e)`, where `src` is an expression resulting in a collection of objects or integers, and (e) is an (expression resulting in an) object or an integer.

Reformulation using Global Constraints:

$$includes(X, y, z) : \begin{cases} count(y, X, c) \\ z = [\![c > 0]\!] \end{cases} \tag{8.15}$$

For this model, we simply count the number of times the element is found in the list. The reified value of the constraint z, is equal to weather or not the count is greater than zero.

For example given $X = [3, 1, 3, 2, d]$ for `src`, and $y = 3$ for `exp`, the count c is equal to the count of y in X, here $c = 2$. Finally the output z constrained to wether or not, $c > 2$.

8.3.3 excludes(e)

Consider the expression `src.excludes(e)`, where `src` is an expression resulting in a collection of objects or integers, and (e) is an (expression resulting in an) object or an integer.

Reformulation using Global Constraints:

$$excludes(X, y, z) : \begin{cases} count(y, X, c) \\ z = [\![c = 0]\!] \end{cases} \tag{8.16}$$

For this model, we simply count the number of times the element is found in the list. The reified value of the constraint z, is equal to weather or not the count is equal to zero.

For example, given $X = [3, 1, 3, 2, d]$ for `src`, and $y = 4$ for `exp`, the count c is equal to the count of y in X, here $c = 0$. Finally the output z is constrained to be equal to wether or not, $c = 0$.

8.3.4 includesAll(e)

Consider the expression `src.includesAll(e)`, where `src` is an expression resulting in a collection of objects or integers, and (e) is (an expression resulting in) a collection of objects or an integers.

Reformulation using Global Constraints:

$$includesAll(X, Y, z) : \begin{cases} \forall y_i \in Y, \\ \quad includes(X, y_i, z'_i) \\ minimum(Z', z) \end{cases} \tag{8.17}$$

For example, given $X = [3, 1, 3, 2, d]$ and $Y = [3, 2]$, we use the previous *includes* model to return an array of booleans $B = [1, 1]$, representing the inclusion of each element. The minimum of B is the truth value of the reified constraint.

8.3.5 excludesAll(e)

Consider the expression `src.excludesAll(e)`, where `src` is an expression resulting in a collection of objects or integers, and (e) is (an expression resulting in) a collection of objects or an integers.

Reformulation using Global Constraints:

$$excludesAll(X, Y, z) : \begin{cases} \forall y_i \in Y, \\ \quad excludes(X, y_i, z'_i) \\ minimum(Z', z) \end{cases} \tag{8.18}$$

For example, given $X = [3, 1, 3, 2, d]$ and $Y = [4, 2]$, we use the previous *includes* model to return an array of booleans $B = [1, 0]$, representing the exclusion of each element. The minimum of B is the truth value of the reified constraint. Here because 2 isn't excluded from X, the minimum found in B is 0, or false.

8.3.6 isEmpty()

Consider the expression `src.isEmpty()`, where `src` is an expression resulting in a collection of objects or integers.

Reformulation using Global Constraints:

$$isEmpty(X, y) : \begin{cases} count(d, X, s) \\ y = [\![s = |X|]\!] \end{cases} \tag{8.19}$$

8.3.7 notEmpty()

Consider the expression `src.notEmpty()`, where `src` is an expression resulting in a collection of objects or integers.

Reformulation using Global Constraints:

$$notEmpty(X, y) : \begin{cases} count(d, X, s) \\ y = [\![s \neq |X|]\!] \end{cases} \tag{8.20}$$

8.3.8 forall(exp)

Consider the expression `src.forall(exp)`, where the source `src` is an expression resulting in a collection of objects or integers and `exp` is an sub-expression of type boolean which applies to a sub-collection of the collection from the source `src`. The expression as a whole resolves to true if all the sub-expressions resolves to true, and false otherwise.

Reformulation using Global Constraints:

$$forall(X, y, exp) : \begin{cases} itpred(X, B, exp) \\ minimum(B, y) \end{cases} \tag{8.21}$$

Our model for `forall` gathers a collection of booleans, and simply textitands across them.

Iterating with predicates

The `forall` expression, along with `exists` and `one`, take an iterable expression which can take several iterators, such as `a,b | pred(a,b)` where `pred` is a predicate applied to a and b.

$$itpred(X, B, exp) : \begin{cases} \forall i, j, k, ... \in [1..|X|] \\ \quad b_{ijk...} \iff (((x_i \neq d) \wedge (x_j \neq d) \wedge ...) \implies exp(x_i, x_j, x_k, ...)) \end{cases} \tag{8.22}$$

In this first equation, we focus on the core of *itpred* associating the input X and the output B. The number of variables in B is equal to $(|X|)^z$ where z is the number of iterators. To index B we'll use z indices, ranging from 1 to $|X|$. Each $b_{ijk...}$, where i,j,k,.. are in [1..|X|], is associated to the combination variables $x_i, x_j, x_k, ..., x_z$. The *itpred* constraint can be understood as a dispatcher, applying the expression *exp* of every combination of variables from X. Each combination of variables is associated to a boolean, which is false only if variables have valid values and the expression upon them is false. To complete the model, lets consider the expression `src.forall(a,b,c| (a<b and b<c) implies (a+b=c))`, `exp` in the above would be replaced with:

$$(x_i < x_j \wedge x_j < x_k) \implies (x_i + x_j = x_k + x_k)$$

8.3.9 exists(exp)

Consider the expression `src.exists(exp)`, where the source `src` is an expression resulting in a collection of objects or integers and `exp` is an sub-expression of type boolean which applies to a sub-collection of the collection from the source `src`. The expression as a whole resolves to true if at least one of the sub-expressions resolves to true, and false otherwise.

Reformulation using Global Constraints:

$$exists(X, y, exp) : \begin{cases} itpred(X, B, exp) \\ maximum(B, y) \end{cases} \tag{8.23}$$

8.3.10 one(exp)

This operation can have at most one iterator variable.

Consider the expression `src.one(exp)`, where the source `src` is an expression resulting in a collection of objects or integers and `exp` is an sub-expression of type boolean which applies to the elements of the collection from the source `src`. The expression as a whole resolves to true if one of the sub-expressions resolves to true, and false otherwise.

Reformulation using Global Constraints:

$$one(X, y, exp) : \begin{cases} itpred(X, B, exp) \\ count(\top, B, y') \\ y = [\![y' = 1]\!] \end{cases} \tag{8.24}$$

8.4 CP Models for OCL Collection Operations

In this section we cover the operations which take collections and return collections.

8.4.1 select(exp)

Consider the expression `src.select(exp)`, where `src` is an expression resulting in a collection and `exp` is a predicate: the output of the operation is the sub-collection of elements for which the predicate is true. This operation can have at most one iterator variable.

$$select(X, Y, exp) : \Big\{ y_i = d - (d - x_i) * [\![exp(x_i)]\!] \tag{8.25}$$

For this model we first assume the value is filtered $y_i = d$, then depending on if the expression is true when applied $exp(x_i)$ we add it back the value x_i, using $-(d - x_i)$.

8.4.2 reject(exp)

Consider the expression `src.reject(exp)`, where `src` is an expression resulting in a collection of objects or integers and `exp` is a predicate: the output of the operation is the sub-collection of elements for which the predicate is false. This operation can have at most one iterator variable.

$$reject(X, Y, exp) : \Big\{ y_i = d - (d - x_i) * [\![\neg exp(x_i)]\!] \tag{8.26}$$

For this model we first assume the value is filtered $y_i = d$, then depending on if the expression is false when applied $exp(x_i)$ we add it back the value x_i, using $-(d - x_i)$.

8.4.3 any()

This operation can have at most one iterator variable. Consider the expression `src.any(exp)`, where `src` is an expression resulting in a collection of objects or integers and `exp` is a predicate: the output of the operation is one of the elements for which the predicate is true. This operation can have at most one iterator variable.

$$any(X, y, exp) : \begin{cases} select(X, Y, exp) \\ asBag(Y, Y') \\ y = y'_1 \end{cases} \tag{8.27}$$

For this model, we simply use the selection model to find good values, and use the as bag model to make sure the first value is valid. Any other way of choosing a valid value could also be chosen.

8.4.4 sortedBy(exp)

This operation can have at most one iterator variable. Consider the expression `src.sortedBy(exp)`, where `src` is an expression resulting in a collection of objects or integers and `exp` results in an integer: the output of the operation is the sequence of elements with respect to the result of the expression. Reformulation using Global Constraints:

$$sortedBy(X, Y, exp) : \begin{cases} stable_keysort(\langle K, X\rangle, \langle K', Y\rangle, 1) \\ k_i = exp(x_i), \forall i \in [1, z] \end{cases} \tag{8.28}$$

For this we simply apply the key sort global constraint, using the result of the expression as key.

8.5 CP Models for OCL Collection Type Casting Operations

To illustrate OCL type casting, consider the following invariant, taken from the Zoo Model used in the experimentation:

```
1 context Cage inv:
2     self.animals.species.asSet().size < 2
```

If `self.animals.species` evaluates to the sequence {1,1,1}, applying `asSet()` yields the set {1}, indicating that the cage contains a single species of animal. In the following, we define CP models to capture such collection type conversions.

Consider an expression of the form `src.asOP()`, where `src` is a collection-valued expression such as `self.attribute`, and `asOP()` denotes a type-casting operation applied to the source collection (e.g., `asSequence()`, `asSet()`, etc.). Let $X = \{x_1, ..., x_z\}$ be the array of variables modeling the values of `src`, and let $Y = \{y_1, ..., y_z\}$ represent the resulting collection after applying `asOP()`.

8.5.1 asBag()

Consider the expression `src.asBag()`, where the result is evaluated as a multiset Y that preserves all values from the source collection X, including repeated elements. Because OCL bags are insensitive to permutations, multiple orderings of the same values are semantically equivalent. To avoid such symmetries in the model, we impose a canonical form by sorting Y in descending order. This also ensures that any dummy values d used to pad the collection appear at the end. For example, given $X = \{1, 2, d, 1\}$, we enforce the canonical bag representation $Y = \{2, 1, 1, d\}$. This transformation is modeled using the global constraint $sort(X, Y)$, which sorts X into Y.

$$asBag(X, Y) : \left\{ sort(X, Y^{\text{rev}}) \right. \tag{8.29}$$

Y^{rev} denotes the reverse of Y, used to enforce descending order.

8.5.2 asSet()

Consider the expression `src.asSet()`. The `asSet()` operation removes duplicate elements from the source collection X while disregarding order. In our encoding, this corresponds to extracting the distinct values from X and placing them into the result array Y in a canonical form. Since the number of unique elements in X is not known beforehand, Y is defined with the same arity as X, and any unused positions are filled with a dummy value d. For instance, given an instantiation $X = \{1, 2, 1, d\}$, the result of `asSet()` would be $Y = \{2, 1, d, d\}$.

$$asSet(X, Y) : \begin{cases} sort(X, S^{\text{rev}}) & \\ X' = S \parallel \{d\} & \\ Y' = Y \parallel \{d\} & \\ p_1 = 1 & \\ \forall i \in]2, z+1] : & p_i = p_{i-1} + [\![x'_{i-1} \neq x'_i]\!] \\ & element(x'_i, Y', p_i) \\ \forall i \in [1, z] : & y_i \geq y_{i+1} \end{cases} \tag{8.30}$$

To enforce the `asSet()` semantics, we first sort the source array X in descending order into an auxiliary array S. We then define an array of position variables P and compute the position p_i of each variable s_i in a new array Y, ensuring that repeated values in S map to the same position. The first occurrence of a new value increments the position counter: $p_i = p_{i-1} + [\![s_{i-1} \neq s_i]\!]$. To support cases where all positions in Y are filled with unique values, we append a dummy value d to S, yielding $X' = S \parallel \{d\}$. In the case where all values in X are distinct (e.g., $X = \{2, 3, 1, 4\}$), the dummy has no room in Y. We resolve this by appending a dummy value to Y as well, forming $Y' = Y \parallel \{d\}$. This dummy will occupy the first unused position in Y, and all subsequent positions are forced to d by a descending sort constraint $y_i \geq y_{i+1}$. The final mapping from positions p_i to Y' is enforced via an $element(c)$onstraint over X' and Y'.

8.5.3 asSequence()

The `asSequence` operation retains all values from the source collection, including duplicates, and reorders them such that all non-dummy values appear first in their original

Index	B	X	**Sorted Index**	B'	Y
1	0	1	1	0	1
2	1	d	3	0	2
3	0	2	5	0	1
4	1	d	2	1	d
5	0	1	4	1	d

Table 8.1: Example of `asSequence()` transformation using stable sort. Dummy values are in red.

relative order, followed by the dummy values. For example, if $X = \{1, d, 2, d, 1\}$, then `asSequence` yields $Y = \{1, 2, 1, d, d\}$. To enforce this transformation, we introduce the following CP model:

$$asSeq_{x2y}(X, Y) : \begin{cases} stable_keysort(\langle B, X\rangle, \langle B', Y\rangle, 1) \\ \quad b_i = [\![x_i = d]\!], \forall i \in [1, z] \\ \quad b'_i = [\![y_i = d]\!], \forall i \in [1, z] \end{cases} \tag{8.31}$$

Here, B and B' are arrays of integer variables of size z, of domain $0, 1$, used as booleans indicating which variables in X and Y are equal to the dummy value d. The $stable_keysort(T, S, k)$ constraint takes a matrix T and produces a sorted matrix S, ordering rows based on the first k columns, which form the sort key. In our case, we construct the matrices $\langle B, X\rangle$ and $\langle B', Y\rangle$, and sort on the first column, which separates dummy and non-dummy values while preserving the original order of the non-dummy elements.

To illustrate, let $X = \{1, d, 2, d, 1\}$, yielding $B = \{0, 1, 0, 1, 0\}$. We apply a stable sort to B, considering the pairs (b_i, x_i), and sorting by the key b_i. This ensures that all 0s (non-dummy values) appear before all 1s (dummy values), and the relative order of elements with the same key (e.g., all 0s) is preserved (see Table **??**). The sorted Boolean array becomes $B' = \{0, 0, 0, 1, 1\}$. Applying the permutation used to sort B to the array X results in $Y = \{1, 2, 1, d, d\}$.

One of the strategies during the search process involves enumerating the variables representing the top-level nodes in the OCL abstract syntax tree (AST). For example, in the expression `src.asSequence().sum()<3`, we explore possible values for `.sum()`, which helps filter the values of `src.asSequence`. To extend this filtering process down to `src`, an additional model is needed to manage the refinement. To filter from Y to X, we use a cumulative constraint, commonly applied in task scheduling. In this approach, we treat

the intervals between values in Y as blocking tasks that prevent certain values from X during scheduling. By scheduling the tasks derived from X around the blocking intervals from Y, we filter down the possible values for X, effectively refining the search space according to the constraints set by Y.

$$asSeq_{y2x}(X,Y) : \begin{cases} & sort(Y,Y') \\ \text{let } T_y & \text{be the set of tasks such that:} \\ \forall i \in [1,z[& : s_i = y'_i + 1 \\ & \quad d_i = \max(0, y'_{i+1} - y'_i - 1) \\ & \quad h_i = z \\ \text{let } T_x & \text{be the set of tasks such that:} \\ \forall i \in [1,z] & : s_i = x_i \\ & \quad d_i = 1 \\ & \quad h_i = 1 \\ & cumulative(T_y \cup T_x, z) \end{cases} \tag{8.32}$$

Equation (8.32) defines how to filter values of X based on the sequence Y using a cumulative constraint:

1. First, Y is sorted into Y' to identify ordered non-dummy values.

2. From Y', we define blocking tasks T_y representing disallowed intervals. Each task (associated to value y'_i in Y'):

   - Starts at $s_i = y'_i + 1$,
   - Has a duration $d_i = \max(0, y'_{i+1} - y'_i - 1)$,
   - Has a height of $h_i = z$, fully consuming the resource and thus excluding X from that interval.

3. For each variable $x_i \in X$, a task is created in T_x starting at x_i, with duration 1 and height 1.

4. The cumulative constraint on $T_y \cup T_x$ ensures tasks from X are only scheduled in the non-blocked intervals.

In Figure 8.9, blocking tasks (highlighted in red and blue) are created from the intervals between values in Y', representing values that are prohibited for X. The white space represents the available slots for scheduling tasks from X. This model effectively restricts the possible values for X by ensuring that certain values, determined by the sorted sequence Y', are "blocked" from being selected, refining the search space. Combining both

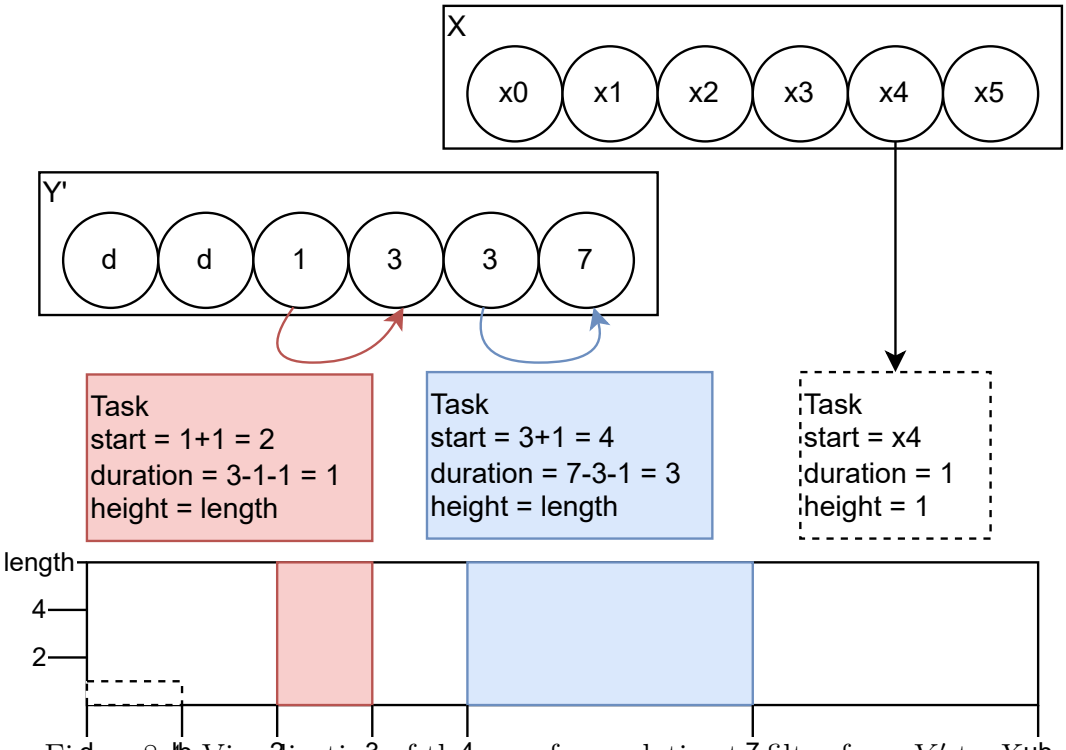


Figure 8.4: Visualization of the use of cumulative to filter from Y' to X

the X to Y and Y to X models give us the complete model for `asSequence`.

$$asSequence(X, Y) : \begin{cases} asSeq_{x2y}(X, Y) \\ asSeq_{y2x}(X, Y) \end{cases} \tag{8.33}$$

8.5.4 asOrderedSet()

Consider the expression `src.asOrderedSet()`. The `asOrderedSet()` operation removes duplicates from X while preserving the relative order of first occurrences. Unused positions in Y are filled with dummy values d. For example if $X = \{1, 2, d, 1\}$, then `asOrderedSet` returns the array $Y = \{1, 2, d, d\}$. To enforce this behavior, we use the following CP

model:

$$asOrdSet(X,Y) : \begin{cases} stable_keysort(< X, Y' >, < S, T >, 1) \\ t_1 = s_1 \\ \forall i \in]1, z] : t_i = \begin{cases} s_i & \text{if } s_i \neq s_{i-1} \\ d & \text{otherwise} \end{cases} \\ asSequence(Y', Y) \end{cases} \tag{8.34}$$

The idea is to sort X into S to group identical values. We build T by keeping the first occurrence of each value in S and replacing subsequent duplicates with the dummy value d. We then invert the sort to obtain Y', restoring the original structure. Finally, we apply `asSequence` to push all dummy values to the end, yielding the final ordered set Y.

Given $X = \{2, 1, 2, 3\}$, sorting yields $S = \{1, 2, 2, 3\}$, filtering gives $T = \{1, 2, d, 3\}$, reversing the sort results in $Y' = \{2, 1, d, 3\}$, and packing dummies yields $Y = \{2, 1, 3, d\}$.

8.5.5 Filtering Dummy Values in OCL Collection Operations

E. Filtering Dummy Values in OCL Collection Operations For many OCL collection operations, the filtering process from X to Y can be enhanced by introducing a dedicated constraint to handle dummy values. This filtering mechanism can be integrated into models such as 8.29, 8.30, 8.33, and 8.34. The filtering approach is inspired by the strategy used in Equation (6.1), employing a *regular* constraint over a masked array:

$$dChannel(X,Y) : \begin{cases} regular(S, NFA) \\ \text{where } s_i = [\![s'_i \neq d]\!], i \in [1, z] \\ \text{with } S' = X \parallel c \parallel Y^{\text{rev}} \end{cases} \tag{8.35}$$

The mask encodes non-dummy values with 1s and dummy values with 0s. The *regular* constraint is applied over the concatenated sequence $S' = X \parallel c \parallel Y^{\text{rev}}$, where c is a counter variable ranging from 0 to z. The non-deterministic finite automaton (NFA), shown in Figure 8.5, ensures that the number of 0s (i.e., dummies) in X is matched by the same number of leading 0s in Y^{rev}.

Given a partial instantiation such as $X = \{x_1, d, x_3, d, x_5\}$, this constraint allows filtering to deduce $Y = \{y_1, y_2, y_3, d, d\}$.

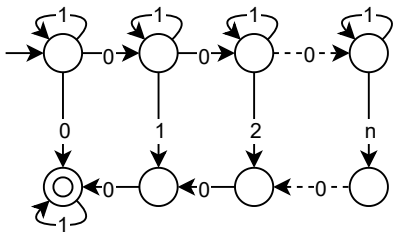


Figure 8.5: Non-Deterministic Finite Automaton accepting strings where Y starts with the same number of 0 found in X.

8.6 CP Models for OCL Sequence and Ordered Set Operations

In this section we'll consider OCL operations which can be applied to ordered collections, namely: sequence and ordered set.

8.6.1 src.prepend(e)

Consider the expression `src.prepend(e)`, where `src` is an expression resulting in a collection of objects or integers and `e` is an expression of resulting in an object or int: the output of the operation is the sequence starting with `e` followed by `src`. Reformulation using Global Constraints:

$$prepend(X, Y, z) : \Big\{ Y = z \ \| \ X \tag{8.36}$$

In this model we simply need to prepend the variable to our array of variables.

8.6.2 src.append(e)

Consider the expression `src.append(e)`, where `src` is an expression resulting in a collection of objects or integers and `e` is an expression of resulting in an object or int: the output of the operation is the sequence starting with `src` followed by `e`.

Reformulation using Global Constraints:

$$append(X, Y, z) : \begin{cases} Y' = X \ \| \ z \\ asSequence(Y', Y) \end{cases} \tag{8.37}$$

In this model, we append the variable to the array of variables, and call the constraint to cast it to a sequence.

8.6.3 src.insertAt(i,e)

Consider the expression `src.insertAt(i,e)`, where `src` is an expression resulting in a collection of objects or integers, `i` is an expression resulting in an integer, and `e` is an expression of resulting in an object or int: the output is the sequence composed of the i-1 first values of `src`, then `e`, followed by the rest of `src`. Reformulation using Global Constraints:

$$insertAt(X, Y, z, p) : \begin{cases} \forall i \in [1..|X|] \;\; p'_i = i + [\![i \geq p]\!] \\ element(x_i, Y, p'_i) \\ element(z, Y, p) \end{cases} \tag{8.38}$$

This model finds the positions of values of X in Y. Their position in Y is unchanged if they're before the included element: $p'_i = i$, and incremented after the included element: $p'_i = i + [\![i \geq p]\!]$.

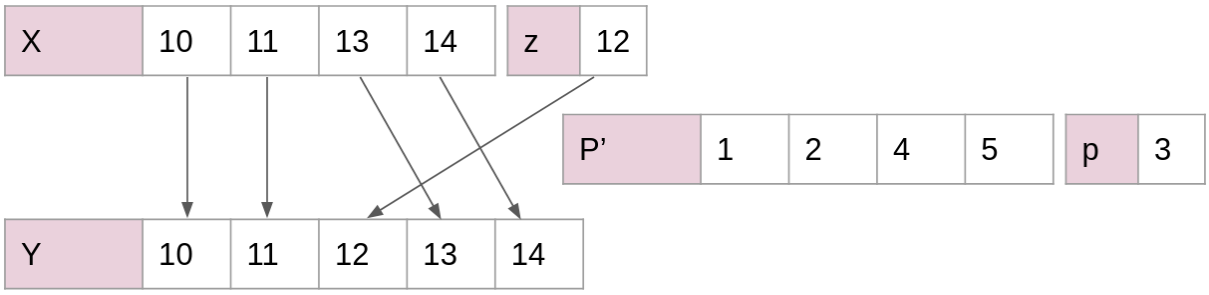


Figure 8.6: example solution for insert at

In this figure we illustrate a solution for this constraint, showing how X is organized into Y around z.

8.6.4 src.subSequence(s,f)

Consider the expression `src.subSequence(s,f)`, where `src` is an expression resulting in a collection of objects or integers and `s,f` are expressions of resulting integers giving the start and finish of the sub-sequence in the sequence: the output is the sub-sequence of `src` starting at s, ending at f. Reformulation using Global Constraints:

$$subSequence(X, Y, s, f) : \begin{cases} X' = d \parallel X \\ p_1 = s + 1 \\ p_i = (s + i) * [\![s + i \leq j]\!] \\ element(y_i, X', p_i) \end{cases} \tag{8.39}$$

In this model we constrain the position of the *output values* in the source variables: $element(y_i, X', p_i)$. Because we generally need to pack the output with dummies, those dummies need a position in the source list. To guarantee the presence of the dummy value in the source, we make X', which a variable instantiated to the dummy value, followed by X.

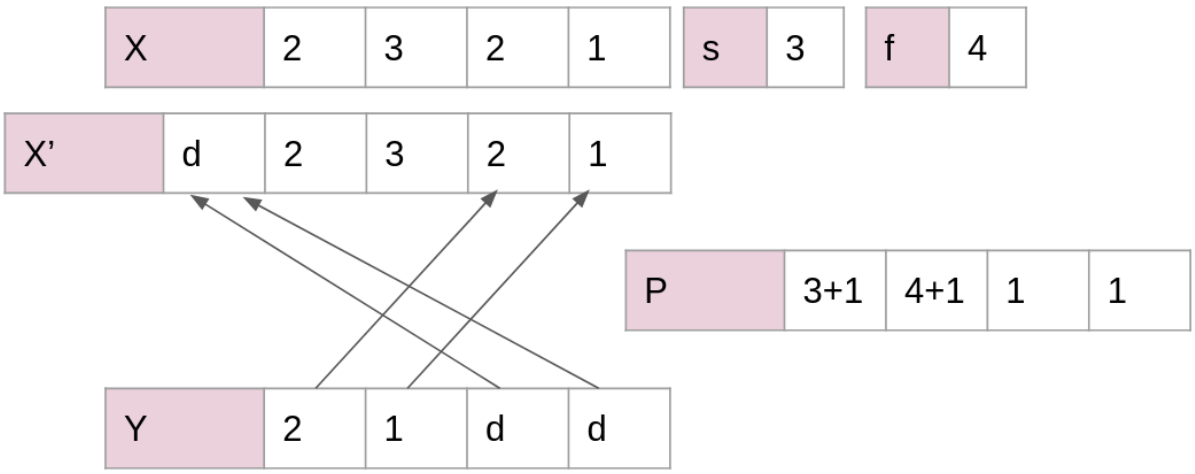


Figure 8.7: example solution for sub-sequence

In this figure, we can see that each value of Y has a position in X'.

8.6.5 src.at(p)

Consider the expression `src.at(p)`, where `src` is an expression resulting in a collection of objects or integers and `p` is an expression of resulting in an integer: the output of this operation is the p-th element of `src`. Reformulation using Global Constraints:

$$at(X, y, p) : \Big\{ element(y, X, p) \tag{8.40}$$

Naturally for this model we simply use the element constraint as intended.

8.6.6 src.indexOf(e)

Consider the expression `src.indexOf(e)`, where `src` is an expression resulting in a collection of objects or integers and `e` is an expression of resulting in an object or an integer: the output of this operation is the position of `e` in `src`. Reformulation using Global Constraints:

$$indexOf(X, y, p) : \Big\{ element(y, X, p) \tag{8.41}$$

For this model, we again employ the element constraint.

8.6.7 src.first()

Consider the expression `src.first()`, where `src` is an expression resulting in a collection of objects or integers: the output is the first element of `src`.

$$first(X, y) : \Big\{ y = x_1 \tag{8.42}$$

For this we operation, we simply need to constrain the output to be equal to the first value of X.

8.6.8 src.last()

Consider the expression `src.last()`, where `src` is an expression resulting in a collection of objects or integers: the output is the last element of `src`.

Reformulation using Global Constraints:

$$last(X, y) : \begin{cases} size(X, p) \\ element(y, X, p) \end{cases} \tag{8.43}$$

Because in out encoding, dummies are packed at the tail end of an array of variables, to get the last value we need to use the size operation to identify the position of the last element.

8.6.9 src.reverse()

Consider the expression `src.reverse()`, where `src` is an expression resulting in a collection of objects or integers: the output of the operations is the sequence in reversed order.

Reformulation using Global Constraints:

$$reverse(X, Y) : \begin{cases} size(X, s) \\ p_i = (s - i) * [\![i \leq s]\!] + i * [\![\neg(i \leq s)]\!] \\ element(x_i, Y, p_i) \end{cases} \tag{8.44}$$

Because our collections are packed with dummies at the tail end, and we wish to keep them there, we reverse the positions values but not dummies. To get the new position of a value, we find its distance to then end, and make that its distance from the start in the new list. If variable holds the dummy value, the value is copied to the same position in the output.

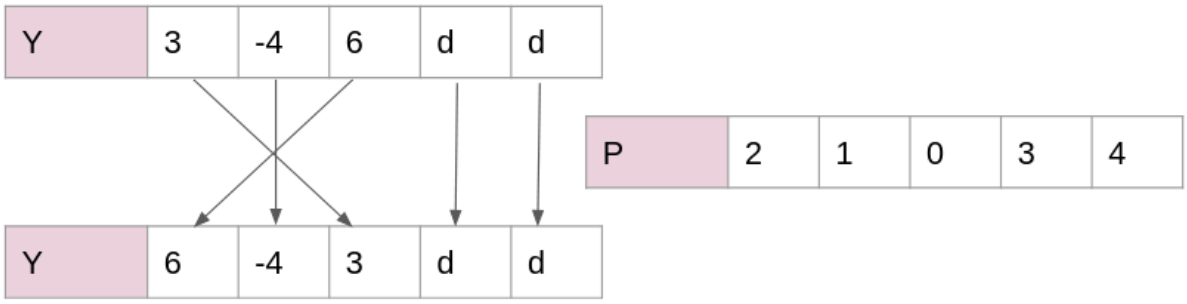


Figure 8.8: example solution for reverse

In this figure we have an example solution, and the intermediate variables giving the positions of values of X in Y.

8.7 CP Models for OCL Set and Bag Operations

In this section we consider operations on unordered collections, namely: bags and sets.

8.7.1 union(exp)

The models for union between sets, and any other combination of bag and set, are very similar. For both of these models, we simply append both arrays of variables, and apply the right type casting operation to ensure conformance.

Consider the expression `src.union(exp)`, where `src` is an expression resulting in a set of objects or integers, and (exp) is (an expression resulting in) a set of objects or an integers: the output of the operation is the set equal to the union of `src` and `exp`. Reformulation using Global Constraints:

$$union_setset(X, Y, Z) : \Big\{ X \ \| \ Y' asSet(Y', Y) \tag{8.45}$$

Consider the expression `src.union(exp)`, where `src` is an expression resulting in a set or bag of objects or integers, and (exp) is (an expression resulting in) a bag of objects or an integers: the output of the operation is the bag equal to the union of `src` and `exp`. Reformulation using Global Constraints:

$$union_Xbag(X, Y, Z) : \Big\{ X \ \| \ Y' asBag(Y', Y) \tag{8.46}$$

8.7.2 Intersection

Consider the expression `src.intersection(exp)`, where `src` is an expression resulting in a set or bag of objects or integers, and (exp) is (an expression resulting in) a set or bag of objects or integers: the output of the operation is the bag equal to the intersection of `src` and `exp`. Reformulation using Global Constraints:

$$intersection(X, Y, Z) : \begin{cases} count(x_i, X, c_i) \\ count(x_i, Y, c'_i) \\ count(x_i, Z, c''_i) \\ c''_i = [\![x_i = d]\!] * a_i + [\![x_i \neq d]\!] * min(c_i, c'_i) \\ a_i \geq max(c_i, c'_i) \\ count(z_i, X, b_i) \\ count(z_i, Y, b'_i) \\ count(z_i, Z, b''_i) \\ b''_i = [\![z_i = d]\!] * a'_i + [\![z_i \neq d]\!] * min(b, b'_i) \\ a'_i \geq max(b_i, b'_i) \end{cases} \tag{8.47}$$

This model works for all combinations of set and bag, and will result in a set if either argument is a set. This model can be divided in two similar parts: towards the output, and towards the input. In the first part, towards the output, we count the occurrences of values of X, in X and Y, and enforce the count of those values in Z to be the minimum of the two. In the second part, towards the input, we count the occurrences of values of Z, in X and Y, and enforce the count of those values in Z to be the minimum of the two. This second part mainly ensures we don't slip in a value instead of a dummy. We don't have a count for the exact number of dummies with this model, but we can enforce a lower bound: at least the greatest number of dummies in X and Y.

In this figure we illustrate a solution and include all the intermediate variables. In the middle we have X, Y, and Z. On the right we have the *towards the output* part, and on the left, the *towards the input* part. Highlighted in blue we have x_2 and its counts in X, Y, and Z. Highlighted in green we have the mirror with z_4 and its counts in X, Y, and Z.

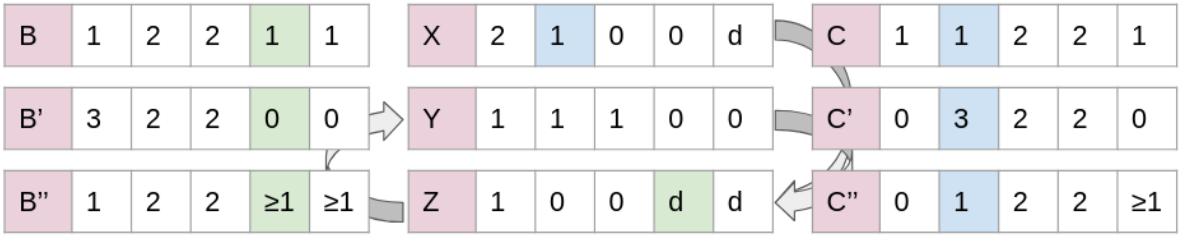


Figure 8.9: Visualization of the use of count to model set and bag intersection

8.7.3 unordered - unordered

Consider the expression `exp1 - exp2`, where `exp1` is an expression resulting in a set or bag of objects or integers, and exp2 is (an expression resulting in) a set or bag of objects or an integers: the output of the operation is the set or bag equal to the difference of `exp1` and `exp2`. The type of the result is the same as `exp1`.

$$diff_set(X, Y, Z) : \begin{cases} count(x_i, Y, s_i) \\ b_i \iff (s_i = 0) \\ z'_i = d - (d - x_i) * [\![b_i]\!] \\ sort(Z', Z^{\text{rev}}) \end{cases} \tag{8.48}$$

This model can be seen as a filter on X for values found in Y, and uses a similar model as that for `X.select(x | Y.count(x)=0)`. And as the OCL describes, this model filters values from X depending on their presence in Y, the result of which is in sorted into Z.

8.7.4 symmetricDifference(exp)

Consider the expression `src.symmetricDifference(exp)`, where `src` is an expression resulting in a set or bag of objects or integers, and exp is (an expression resulting in) a set or bag of objects or an integers: the output of the operation is the set or bag equal to the symmetric difference of `src` and `exp`. The result of this operation is a set, if both

arguments are sets.

$$symdiff_set(X, Y, Z) : \begin{cases} diff_set(X, Y, S) \\ diff_set(Y, X, T) \\ C = S \ \| \ T \\ sort(C, Z^{\text{rev}}) \end{cases} \tag{8.49}$$

This model simply reuses the model for difference, filtering for both X and Y. The results, S and T, are then combined and sorted into the output Z.

8.8 CP Models for OCL Bag Operations

Conclusion

Lorem ipsum dolor sit amet, « consectetuer » adipiscing elit. Maecenas fermentum, elit non lobortis cursus, orci velit suscipit est, id mollis turpis mi eget orci. Ut aliquam sollicitudin metus. Mauris at sapien sed sapien congue iaculis. Nulla lorem urna, bibendum id, laoreet iaculis, nonummy eget, massa. Phasellus ullamcorper commodo velit. Class aptent taciti sociosqu ad litora torquent per « conubia nostra », per inceptos hymenaeos. Phasellus est. Maecenas felis augue, gravida quis, porta adipiscing, iaculis vitae, felis. Nullam ipsum. Nulla a sem ac leo fringilla mattis. Phasellus egestas augue in sem. Etiam ac enim non mauris ullamcorper scelerisque. In wisi leo, malesuada vulputate, tempor sit amet, facilisis vel, velit. Mauris massa est, sodales placerat, luctus id, hendrerit a, urna. Nullam eleifend pede eget odio. Duis non erat. Nullam pellentesque.

> Maître Corbeau, sur un arbre perché,
> Tenait en son bec un fromage.
> Maître Renard, par l'odeur alléché,
> Lui tint à peu près ce langage :
> « Hé ! bonjour, Monsieur du Corbeau.
> Que vous êtes joli ! que vous me semblez beau !
> Sans mentir, si votre ramage
> Se rapporte à votre plumage,
> Vous êtes le Phénix des hôtes de ces bois. »

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Maecenas fermentum, elit non lobortis cursus, orci velit suscipit est, id mollis turpis mi eget orci. Ut aliquam sollicitudin metus. Mauris at sapien sed sapien congue iaculis. Nulla lorem urna, bibendum id, laoreet iaculis, nonummy eget, massa[1]. Phasellus ullamcorper commodo velit. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Phasellus est. Maecenas felis augue, gravida quis, porta adipiscing, iaculis vitae, felis. Nullam ipsum. Nulla a sem ac leo fringilla mattis. Phasellus egestas augue in sem. Etiam ac enim non mauris ullamcorper scelerisque. In wisi leo, malesuada vulputate, tempor sit amet, facilisis vel, velit. Mauris massa est, sodales placerat, luctus id, hendrerit a, urna. Nullam eleifend pede eget odio. Duis non erat. Nullam pellentesque.

[1] **Pierre1901**.

Première section de l'intro

Lorem ipsum dolor sit amet, « consectetuer » adipiscing elit. Maecenas fermentum, elit non lobortis cursus, orci velit suscipit est, id mollis turpis mi eget orci. Ut aliquam sollicitudin metus. Mauris at sapien sed sapien congue iaculis. Nulla lorem urna, bibendum id, laoreet iaculis, nonummy eget, massa. Phasellus ullamcorper commodo velit. Class aptent taciti sociosqu ad litora torquent per « conubia nostra », per inceptos hymenaeos. Phasellus est. Maecenas felis augue, gravida quis, porta adipiscing, iaculis vitae, felis. Nullam ipsum. Nulla a sem ac leo fringilla mattis. Phasellus egestas augue in sem. Etiam ac enim non mauris ullamcorper scelerisque. In wisi leo, malesuada vulputate, tempor sit amet, facilisis vel, velit. Mauris massa est, sodales placerat, luctus id, hendrerit a, urna. Nullam eleifend pede eget odio. Duis non erat. Nullam pellentesque.

Une boite magique :

Titre de la boite

Praesent placerat, ante at venenatis pretium, diam turpis faucibus arcu, nec vehicula quam lorem ut leo. Sed facilisis, augue in pharetra dapibus, ligula justo accumsan massa, eu suscipit felis ipsum eget enim.

Laoreet iaculis, nonummy eget, massa. Phasellus ullamcorper commodo velit. Class aptent taciti sociosqu ad litora torquent per « conubia nostra », per inceptos hymenaeos. Phasellus est. Maecenas felis augue, gravida quis, porta adipiscing, iaculis vitae, felis. Nullam ipsum. Nulla a sem ac leo fringilla mattis. Phasellus egestas augue in sem. Etiam ac enim non mauris ullamcorper scelerisque. In wisi leo, malesuada vulputate, tempor sit amet, facilisis vel, velit. Mauris massa est, sodales placerat, luctus id, hendrerit a, urna. Nullam eleifend pede eget odio. Duis non erat. Nullam pellentesque.

Une boite simple :

Mauris lorem quam, tristique sollicitudin egestas sed, sodales vel leo. In hac habitasse platea dictumst. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed sed lorem lacus, at venenatis elit. Pellentesque nisl arcu, blandit ac eleifend non, sodales a quam.

Laoreet iaculis, nonummy eget, massa. Phasellus ullamcorper commodo velit. Class aptent taciti sociosqu ad litora torquent per « conubia nostra », per inceptos hymenaeos. Phasellus est. Maecenas felis augue, gravida quis, porta adipiscing, iaculis vitae, felis. Nullam ipsum. Nulla a sem ac leo fringilla mattis. Phasellus egestas augue in sem. Etiam ac enim non mauris ullamcorper scelerisque. In wisi leo, malesuada vulputate, tempor sit amet, facilisis vel, velit. Mauris massa est, sodales placerat, luctus id, hendrerit a, urna. Nullam eleifend pede eget odio. Duis non erat. Nullam pellentesque.

COLLEGES BRETAGNE LOIRE | SCIENCES POUR L'INGENIEUR ET LE NUMERIQUE

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Titre : Exploration d'Ensembles de Modèles II

Mot clés : Ingénierie Dirigée par les Modèles, Programmation par Contraintes, Exploration d'Ensembles de Modèles

Résumé : Eius populus ab incunabulis primis ad usque pueritiae tempus extremum, quod annis circumcluditur fere trecentis, circummurana pertulit bella, deinde aetatem ingressus adultam post multiplices bellorum aerumnas Alpes transcendit et fretum, in iuvenem erectus et virum ex omni plaga quam orbis ambit inmensus, reportavit laureas et triumphos, iamque vergens in senium et nomine solo aliquotiens vincens ad tranquilliora vitae discessit. Hoc inmaturo interitu ipse quoque sui pertaesus excessit e vita aetatis nono anno atque vicensimo cum quadriennio imperasset. natus apud Tuscos in Massa Veternensi, patre Constantio Constantini fratre imperatoris, matreque Galla. Thalassius vero ea tempestate praefectus praetorio praesens ipse quoque adrogantis ingenii, considerans incitationem eius ad multorum augeri discrimina, non maturitate vel consiliis mitigabat, ut aliquotiens celsae potestates iras principum molliverunt, sed adversando iurgandoque cum parum congrueret, eum ad rabiem potius evibrabat, Augustum actus eius exaggerando creberrime docens, idque, incertum qua mente, ne lateret adfectans. quibus mox Caesar acrius efferatus, velut contumaciae quoddam vexillum altius erigens, sine respectu salutis alienae vel suae ad vertenda opposita instar rapidi fluminis irrevocabili impetu ferebatur. Hae duae provinciae bello quondam piratico catervis mixtae praedonum.

Title: Model Space Exploration II

Keywords: Model Driven Engineering, Constraint Programming, Model Space Exploration

Abstract: Eius populus ab incunabulis primis ad usque pueritiae tempus extremum, quod annis circumcluditur fere trecentis, circummurana pertulit bella, deinde aetatem ingressus adultam post multiplices bellorum aerumnas Alpes transcendit et fretum, in iuvenem erectus et virum ex omni plaga quam orbis ambit inmensus, reportavit laureas et triumphos, iamque vergens in senium et nomine solo aliquotiens vincens ad tranquilliora vitae discessit. Hoc inmaturo interitu ipse quoque sui pertaesus excessit e vita aetatis nono anno atque vicensimo cum quadriennio imperasset. natus apud Tuscos in Massa Veternensi, patre Constantio Constantini fratre imperatoris, matreque Galla. Thalassius vero ea tempestate praefectus praetorio praesens ipse quoque adrogantis ingenii, considerans incitationem eius ad multorum augeri discrimina, non maturitate vel consiliis mitigabat, ut aliquotiens celsae potestates iras principum molliverunt, sed adversando iurgandoque cum parum congrueret, eum ad rabiem potius evibrabat, Augustum actus eius exaggerando creberrime docens, idque, incertum qua mente, ne lateret adfectans. quibus mox Caesar acrius efferatus, velut contumaciae quoddam vexillum altius erigens, sine respectu salutis alienae vel suae ad vertenda opposita instar rapidi fluminis irrevocabili impetu ferebatur. Hae duae provinciae bello quondam piratico catervis mixtae praedonum.